

2025

Système de gestion d'un hôpital

RIME MOTAI

11/04/2025

I. Étude de domaine : Système de gestion d'un hôpital

1. Problématique

La gestion administrative et médicale des hôpitaux repose encore sur des méthodes manuelles ou des systèmes non centralisés dans plusieurs établissements, ce qui entraîne :

- Des pertes de temps dans le traitement des dossiers médicaux.
- Un manque de traçabilité dans les soins administrés aux patients.
- Une surcharge de travail pour le personnel administratif et médical.
- Des difficultés à accéder rapidement à l'historique médical d'un patient.

2. Solutions proposées

Pour répondre à ces problèmes, on propose de développer un **système informatique de gestion hospitalière (SIH)** basé sur la technologie **Java**. Ce système permettra :

- **La gestion des patients** : enregistrement, consultation du dossier médical, historique des soins.
- **La gestion des médecins** : planning, spécialités, affectations.
- **La gestion des consultations** : prescriptions, traitements.
- **L'authentification sécurisée** : accès différencié selon les rôles (Réceptionniste).
- **Une base de données relationnelle (MySQL)** pour stocker toutes les informations de façon centralisée.

3. Objectifs du projet

1. **Automatiser les processus de gestion hospitalière** afin de réduire les tâches manuelles répétitives.
2. **Améliorer la qualité des soins** par un accès rapide et structuré à l'information médicale.
3. **Assurer la traçabilité** de toutes les interventions médicales.
4. **Renforcer la communication** entre les différents services de l'hôpital.
5. **Offrir une interface intuitive** pour les utilisateurs (interface Swing).
6. **Sécuriser les données médicales** et assurer la confidentialité via un système de rôles et de permissions.

II. 🔍 Analyse :

1. 📌 Étude de l'existant :

Dans de nombreux hôpitaux (notamment les structures publiques ou semi-informatisées), le système de gestion actuel repose souvent sur :

- Des **documents papier** pour les dossiers des patients.
- Des **fichiers Excel** ou des logiciels basiques pour les rendez-vous.
- Une **saisie manuelle** des comptes-rendus de consultations.
- Un **manque d'intégration** entre les services (réception, consultation, laboratoire).
- Peu ou pas de **traçabilité** numérique des soins administrés.

2. 🛠 Critique de l'existant :

Inconvénients majeurs :

- ❌ **Perte de temps** dans la recherche des dossiers.
- ❌ **Risque d'erreurs** de saisie ou de perte de documents.
- ❌ **Confidentialité faible** des données médicales.
- ❌ **Pas de sauvegarde centralisée** ni d'accès distant aux données.

3. ✅ Solutions proposées :

- Développement d'un **Système d'Information Hospitalier (SIH)** en Java avec interface Swing .
- Centralisation des données dans une **base MySQL**.
- Mise en place d'un **accès sécurisé** avec rôle (réceptionniste).
- Intégration d'une interface pour la **réceptionniste** (actrice principale côté accueil).

4. 🧑 Exigences du système :

👤 Acteur : Réceptionniste :

Responsabilités :

- Enregistrer un nouveau patient et son diagnostic.
- Vérifier l'historique des patients.
- Modifier les informations des patients.
- Ajouter des informations à propos de l'hôpital.

Exigences fonctionnelles :

R1 : Authentification nécessaire pour accéder à l'application.

R2 : Possibilité d'ajouter un nouveau patient.

R3 : Rechercher le patient et ajouter son diagnostic.

R4 : Vérifier l'historique des patients ajoutés.

R5 : Modifier les informations des patients ajoutés.

Exigences de qualité :

R6 : L'application doit être conviviale et intuitive pour garantir une expérience utilisateur positive.

R7 : L'application doit être utilisable par l'utilisateur sans grandes connaissances en informatique.

Exigences de performance :

R8 : Répondre à une requête (recherche patient) en moins de 2 secondes.

R9 : Gérer une base de données de plusieurs patients sans perte de performance.



Powered By Visual Paradigm Community Edition

Figure1 : Diagramme de critères

III. Conception :

1. Introduction :

Dans la conception, je vais entamer le processus de conception en appuyant sur les résultats de la phase d'analyse pour modéliser les objectifs à atteindre. J'utiliserais le langage UML (Unified Modeling Language), conçu pour la visualisation, la spécification et la construction des systèmes logiciels. Cette phase est la plus complexe du projet et vise principalement à préciser le modèle afin qu'il puisse être implémenté avec les composantes de l'architecture. Une démarche structurée sera adoptée pour assurer une conception de qualité et une implémentation efficace.

2. UML :

a. Définition:

UML (Unified Modeling Language) est un langage unifié de modélisation orienté objet. Il s'agit d'un langage graphique permettant de spécifier, de construire, de visualiser et de documenter les aspects statiques et dynamiques d'un système logiciel.

b. Niveau Applicatif:

Pour répondre aux besoins identifiés lors de l'analyse, j'utiliserais plusieurs types de diagrammes UML. La démarche adoptée inclut les étapes suivantes:

1. Identification des Cas d'Utilisation:

- Définition des cas d'utilisation qui décrivent les interactions entre l'utilisateur et le système.
- Élaboration des diagrammes de cas d'utilisation détaillés.

2. Diagrammes de Séquence et d'Activité:

- Utilisation des diagrammes de séquence pour modéliser les Interactions entre l'acteurs et le système.
- Utilisation des diagrammes d'activité pour décrire les processus métiers et les flux de travail.

3. Diagramme de Classe:

- Définition des classes par synthèse des diagrammes de séquence et d'activité.
- Élaboration du diagramme de classe, détaillant les classes et leurs relations.

c. Niveau de Données :

Ce niveau concerne l'organisation conceptuelle, logique et physique des données manipulées par l'application. À partir des données identifiées lors de l'analyse, je définirais les classes significatives pour élaborer la conception de la base de données.

3. Démarche de Conception :

La démarche adoptée pour la conception de l'application repose sur plusieurs étapes clés:

a. Identification des Acteurs et des Besoins:

- Identification des utilisateurs du système et de leurs besoins spécifiques.

b. Identification et Représentation des Cas d'Utilisation:

- Définition des cas d'utilisation et élaboration des diagrammes correspondants.

c. Élaboration des Diagrammes de Séquence:

- Formalisation des interactions entre les acteurs et le système au travers des diagrammes de séquence.

d. Élaboration des Diagrammes d'Activité:

Description des processus métiers et des flux de travail à l'aide des diagrammes d'activité.

e. Élaboration des Diagrammes de Classe:

Définition des classes et de leurs relations pour structurer le modèle de données de l'application.

d. Élaboration des Diagrammes d'Interaction:

Modélisation des interactions entre les différentes entités du système.

4. Représentation de la méthodologie RUP :

La figure ci-dessous illustre la démarche de modélisation choisie pour concevoir notre application.

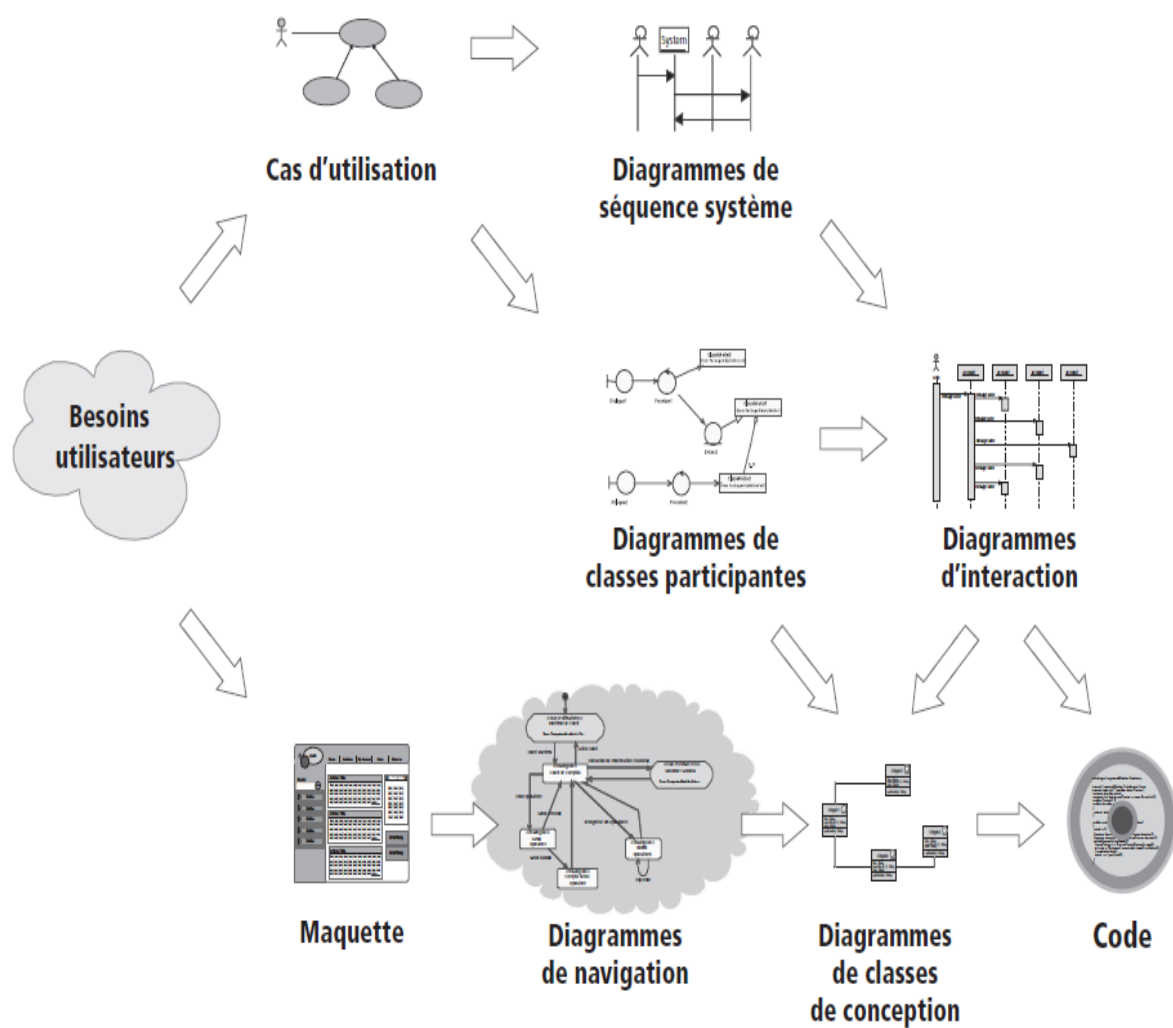


Figure 2 :Cycle de modelisation de l'application

a. Diagramme de Cas d'Utilisation :

Les diagrammes de cas d'utilisation permettent de représenter les Interactions entre les acteurs (utilisateurs, systèmes externes) et le système étudié. Ils illustrent les différentes fonctionnalités offertes par le système et montrent comment les acteurs interagissent avec celui-ci pour atteindre leurs objectifs. Pour réaliser un diagramme de cas d'utilisation, nous avons besoin d'acteurs représentés par des figures humaines et des cas d'utilisation représentés par des ellipses.

b. Identification des Acteurs :

Les acteurs sont les utilisateurs ou entités qui interagissent avec le système. Voici les principaux acteurs identifiés:

- **Réceptionniste** : Personne qui gère les patients et organise leurs informations.



Figure 3 : Diagramme de cas d'utilisation générale

c. Identification des acteurs :

UC1 : Ajouter un nouveau patient.

UC2 : Ajouter un diagnostic.

UC3 : Gérer l'historique de l'hôpital.

UC4 : Modifier les informations d'un un patient.

UC5 : Authentification.

d. Classement des cas d'utilisation :

Exigence Fonctionnelle	Intention d'acteur	Acteur
R1 : Authentification nécessaire pour accéder à l'application.	Effectuer une connexion avec l'application	Réceptionniste
R2 : Possibilité d'ajouter un nouveau patient.	Effectuer l'ajout d'un patient	Réceptionniste
R3 : Rechercher le patient et ajouter son diagnostic.	Ajouter le diagnostic	Réceptionniste
R4 : Vérifier l'historique des patients ajoutés.	Consulter l'historique	Réceptionniste
R5 : Modifier les informations des patients ajoutés.	Rechercher le patient et modifier ses infos	Réceptionniste

Tableau1 : regroupements des exigences

e. Diagramme de cas d'utilisation :

Le schéma ci-dessous représente le diagramme de cas d'utilisation

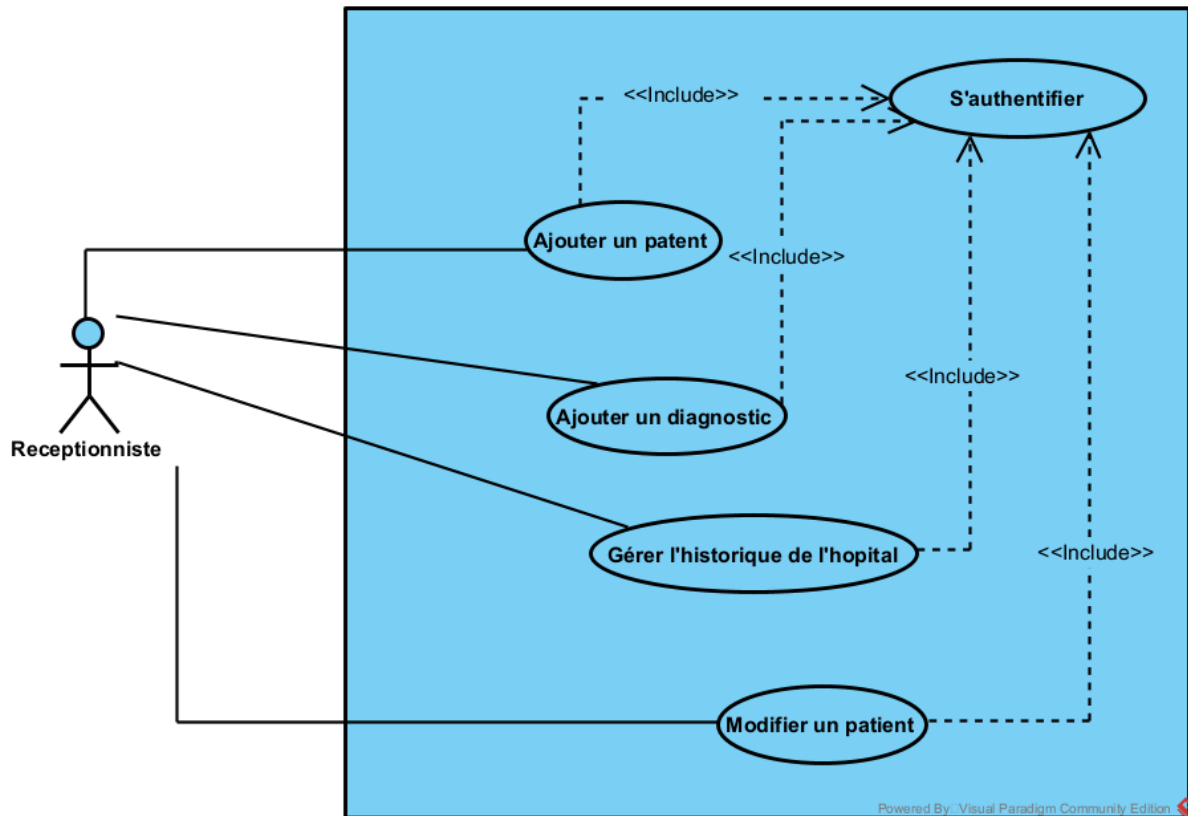


Figure4 : Diagramme de cas d'utilisation

f. Classement des cas d'utilisation :

Pour classer les cas d'utilisation, il est important de décider qu'ils sont les cas les plus prioritaires et qu'il sont les cas qui présente de grands risques techniques.

Cas d'utilisation	Priorité	Risque
UC1 : Ajouter un nouveau patient.	Moyenne	Moyen
UC2 : Ajouter un diagnostic.	Haute	Haut
UC3 : Gérer l'historique de l'hôpital.	Moyenne	Moyen
UC4 : Modifier les informations d'un patient.	Moyenne	Moyen
UC5 : Authentification	Haute	Haut

Tableau2 : Classement des cas

g. Planification du projet

Les cas d'utilisations qui ont une priorité élevée et un risque élevé sont placés dans les premières itérations , on obtient ainsi le classement suivant :

Cas d'utilisation	Priorité	Risque	Itération
UC1 : Ajouter un nouveau patient.	Moyenne	Moyen	1
UC2 : Ajouter un diagnostic.	Haute	Haut	2
UC3 : Gérer l'historique de l'hôpital.	Moyenne	Moyen	3
UC4 : Modifier les informations d'un un patient.	Moyenne	Moyen	4
UC5 : Authentification	Haute	Haut	5

Tableau 3 : Planification du projet

h. Identification des classes :

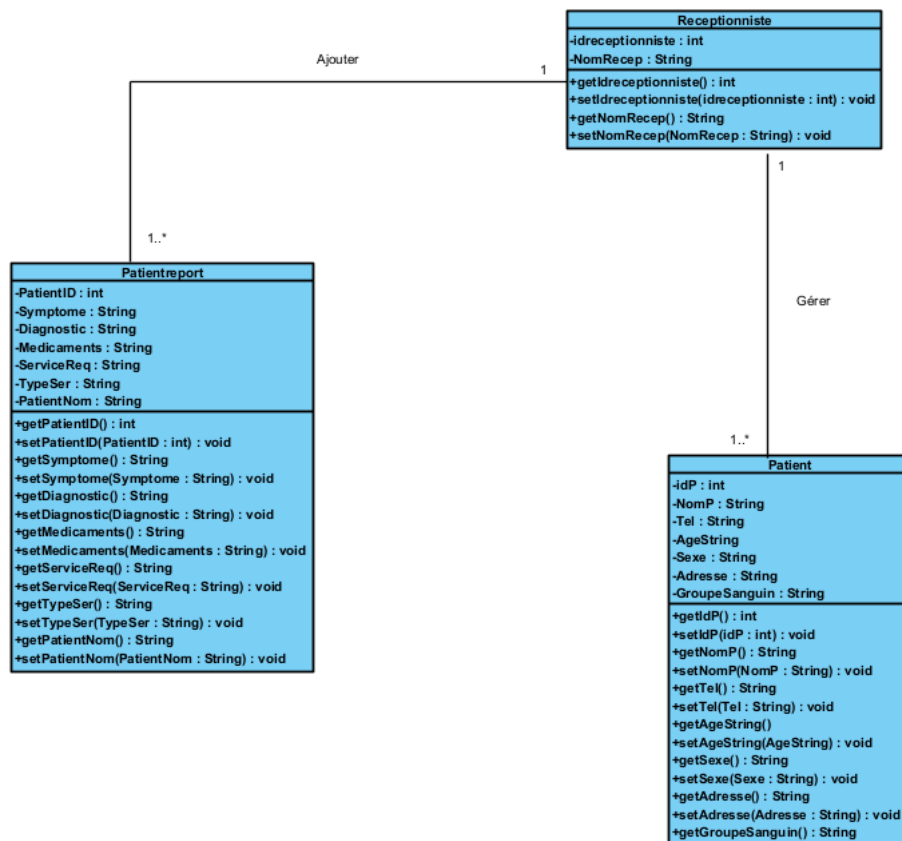


Figure5 : Diagramme de classes

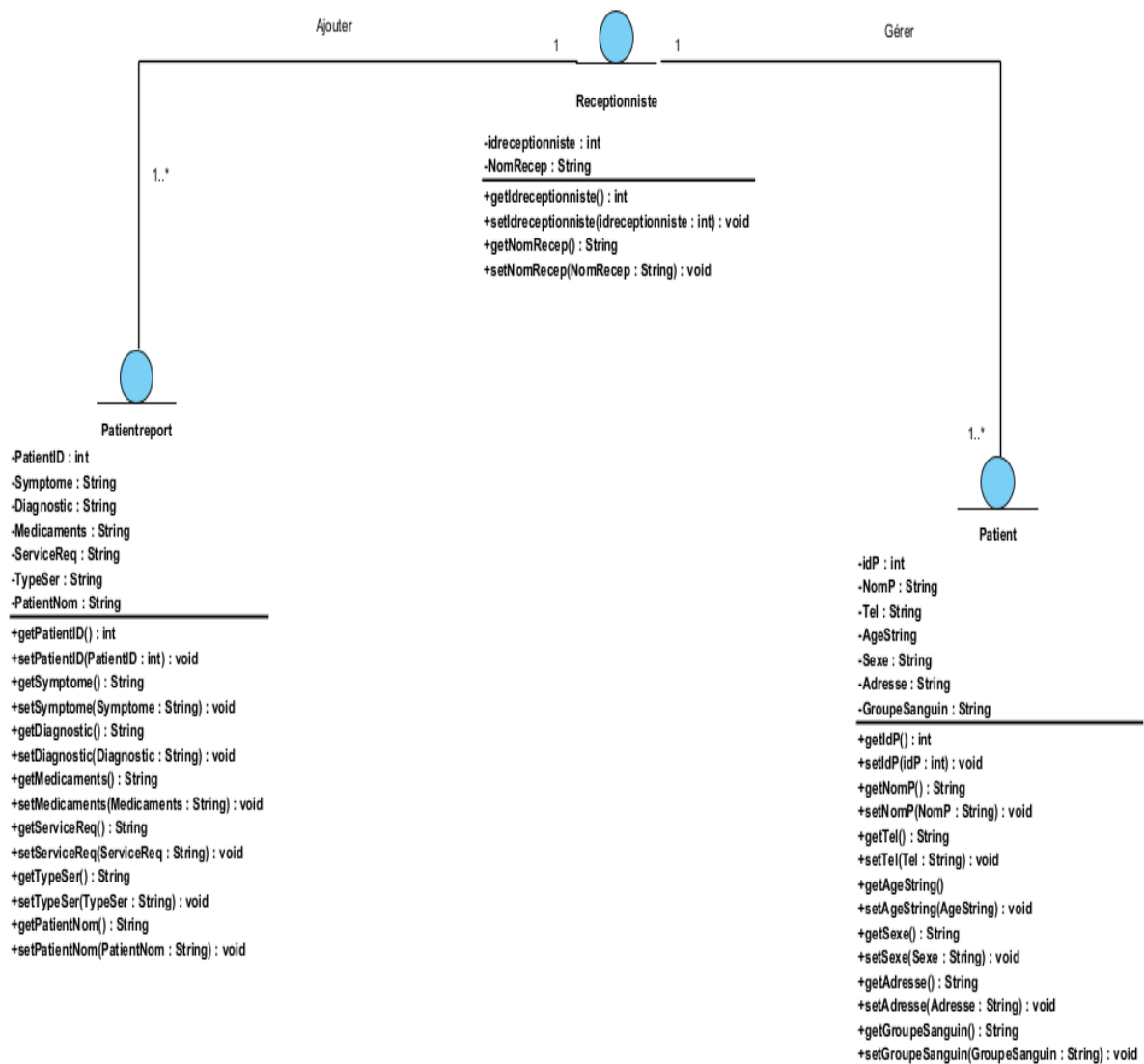


Figure6 : Diagramme de classes participantes

5.Représentation graphique des itérations :

a. Iteration1 : S'authentifier

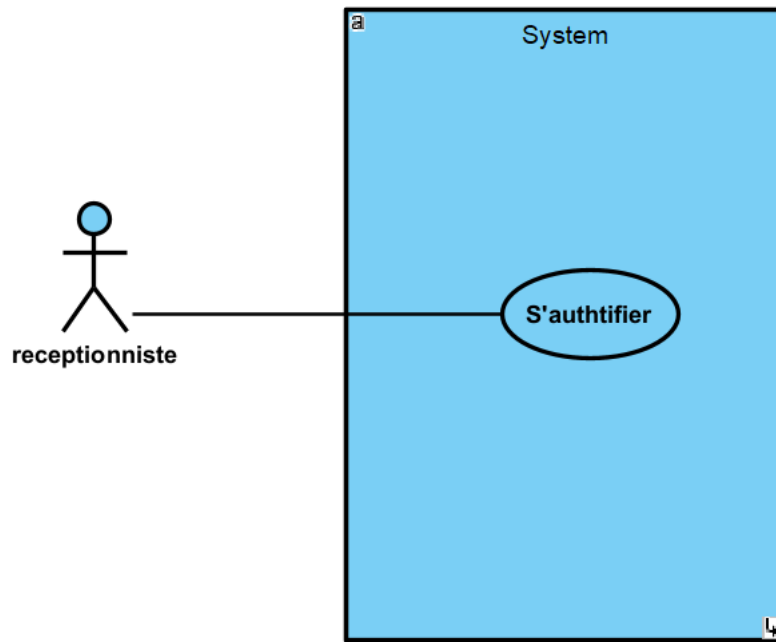
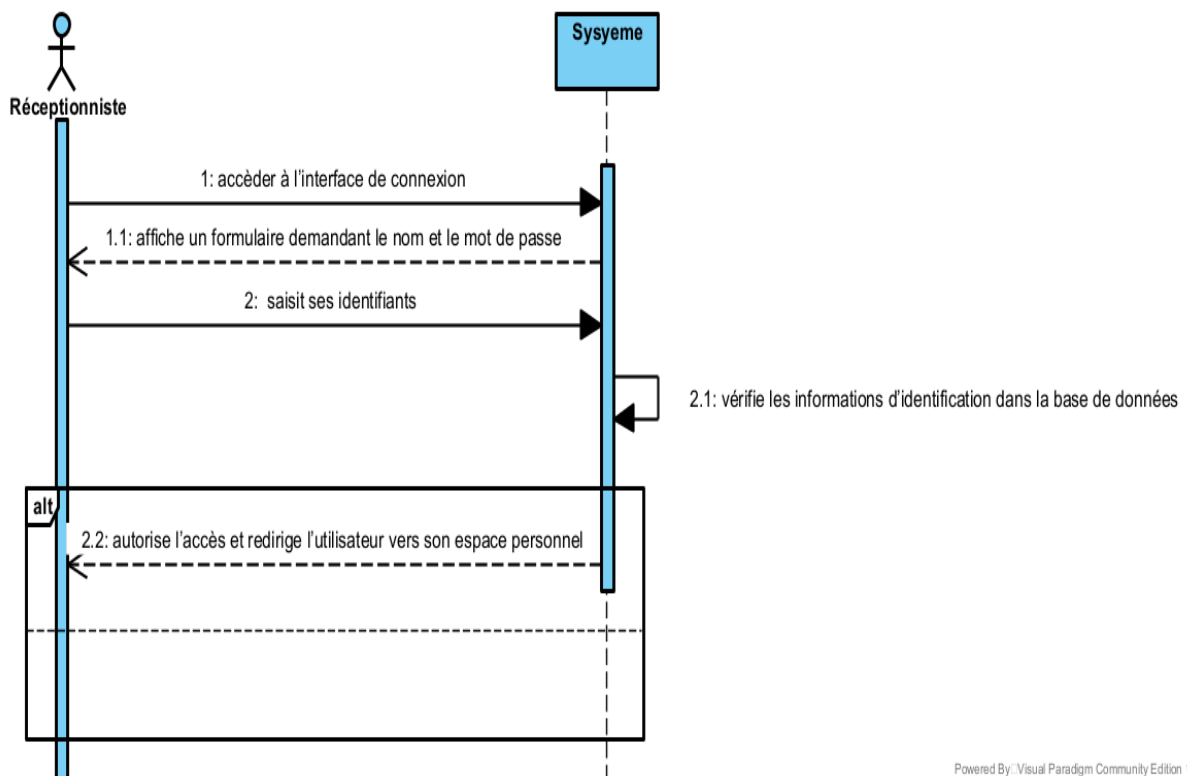


Figure7 : Diagramme de cas d'utilisation Iteration1

1) Diagramme de séquence :



Powered By: Visual Paradigm Community Edition

Figure8 : Diagramme de séquence Iteration1

2) Diagramme d'activité :

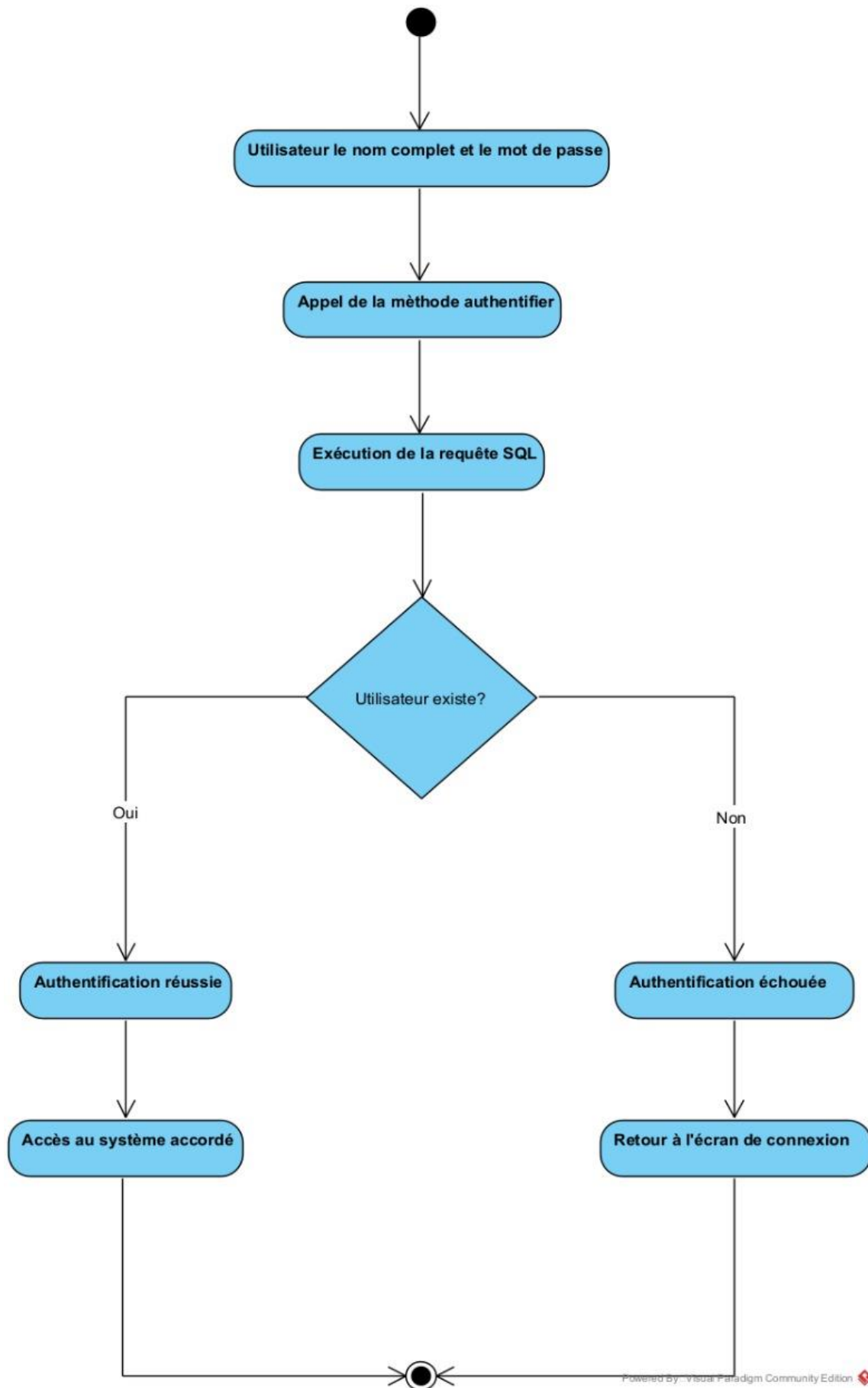


Figure9 : Diagramme d'activité Iteration1

3) Diagramme de classes :

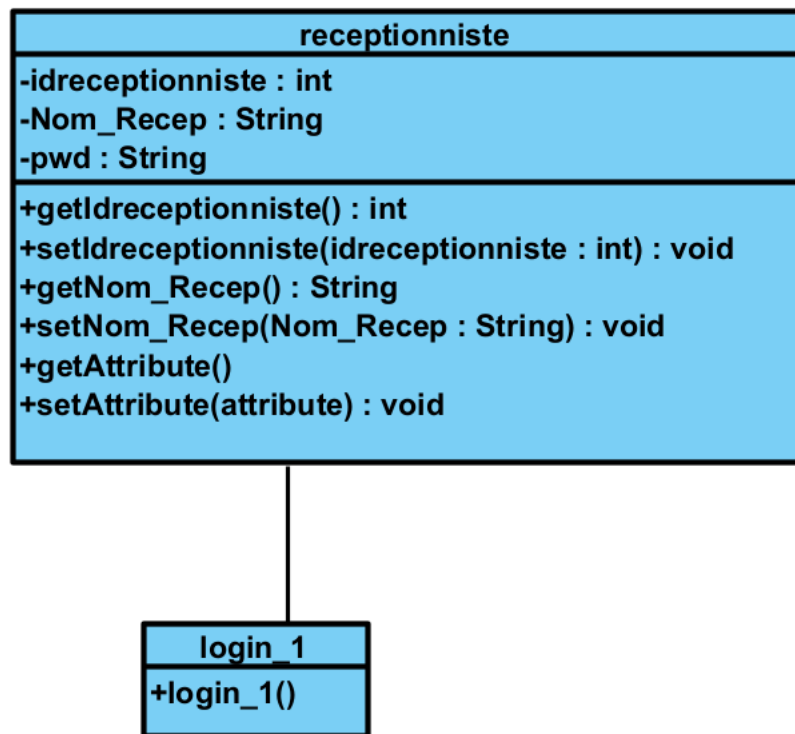


Figure10 : Diagramme de classes Iteration1

4) Diagramme de classes participantes :

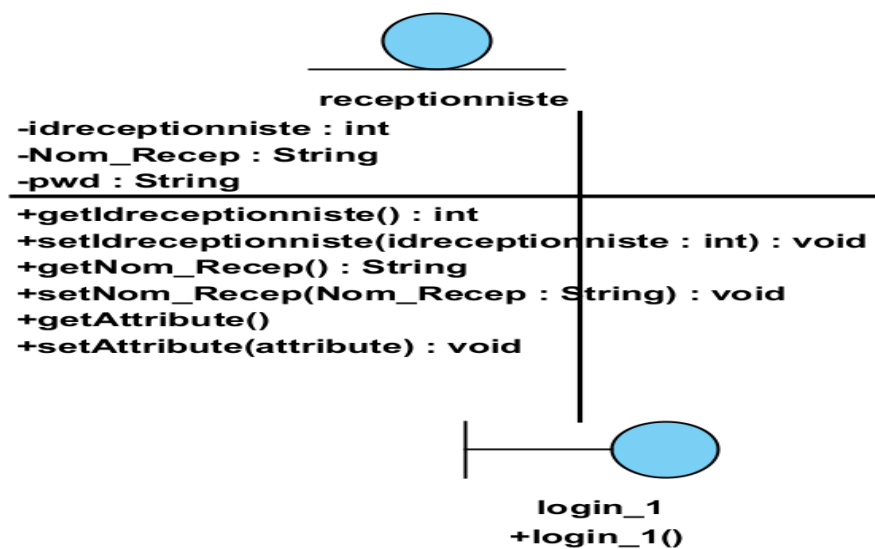


Figure11 : Diagramme de classes participantes Iteration1

b. itération 2 : Ajouter un nouveau patient.

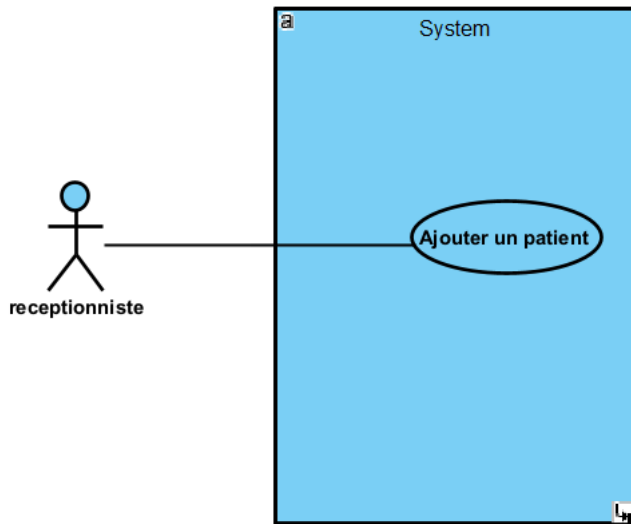


Figure12 : Diagramme de cas d'utilisation Iteration2

1) Diagramme de séquence :

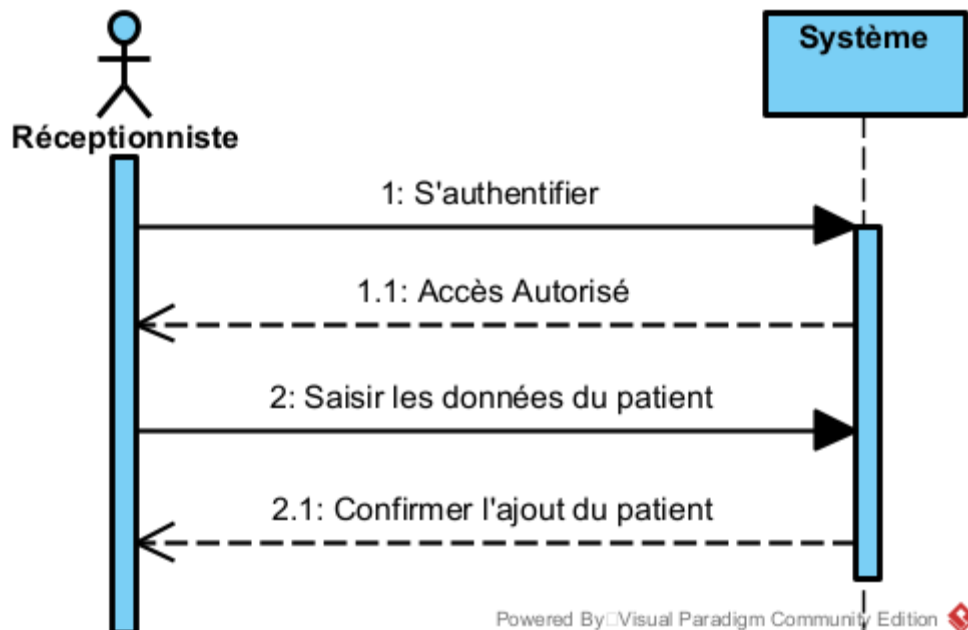


Figure13 : Diagramme de séquence Iteration2

2) Diagramme d'activité :

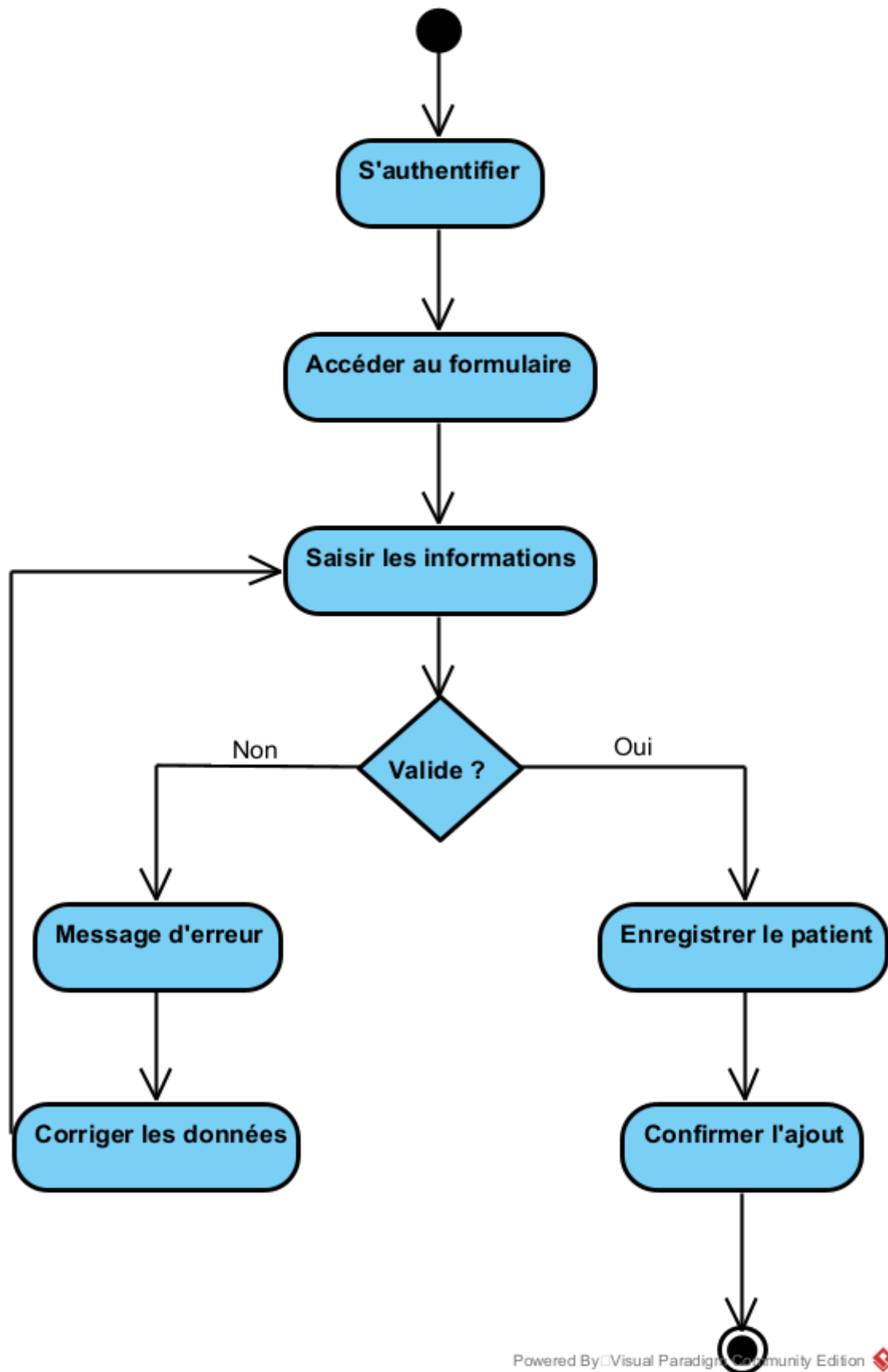


Figure14 : Diagramme d'activité Iteration2

3) Diagramme de classes :

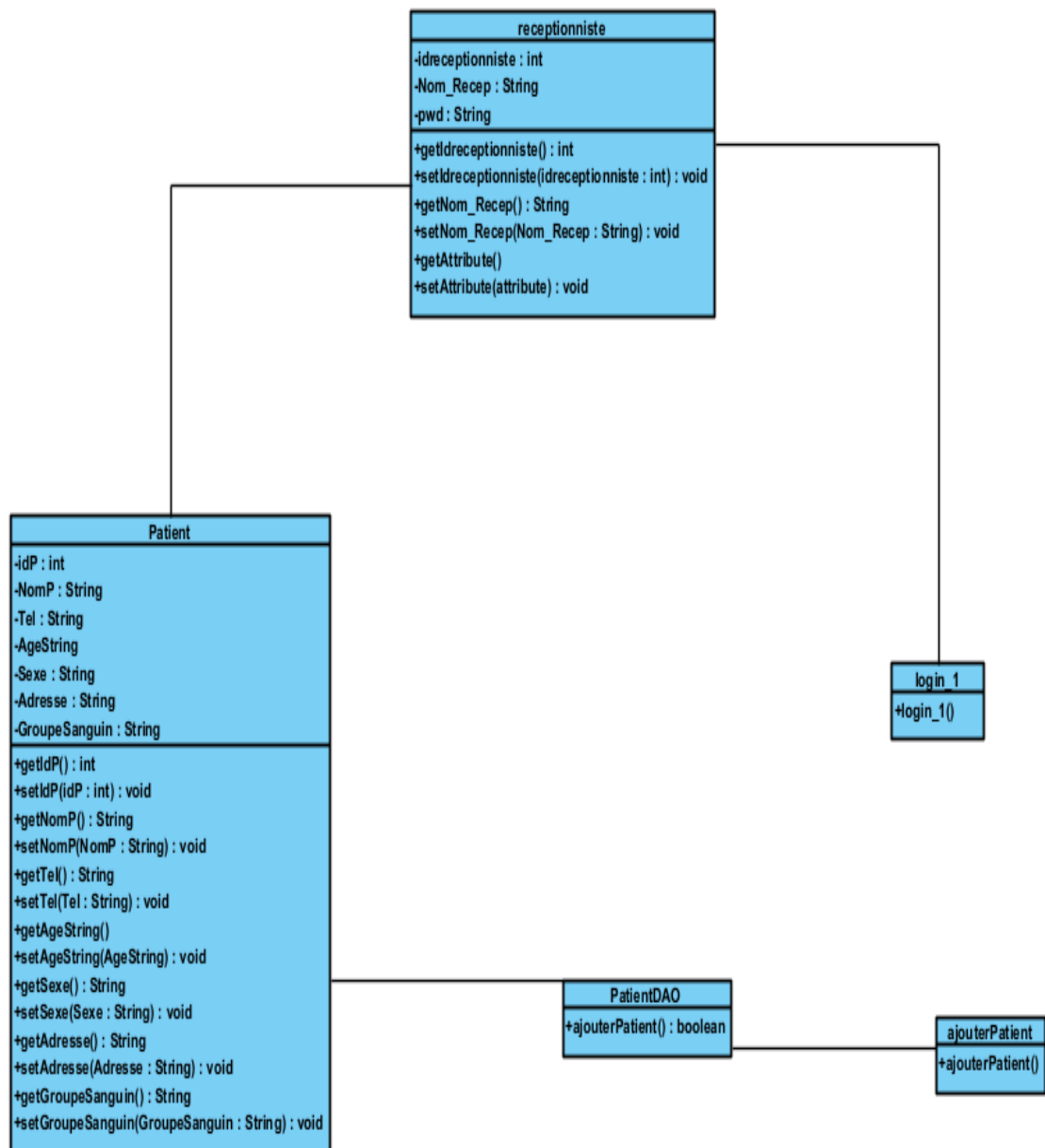


Figure15 : Diagramme de classes Iteration2

5) Diagramme de classes participantes :

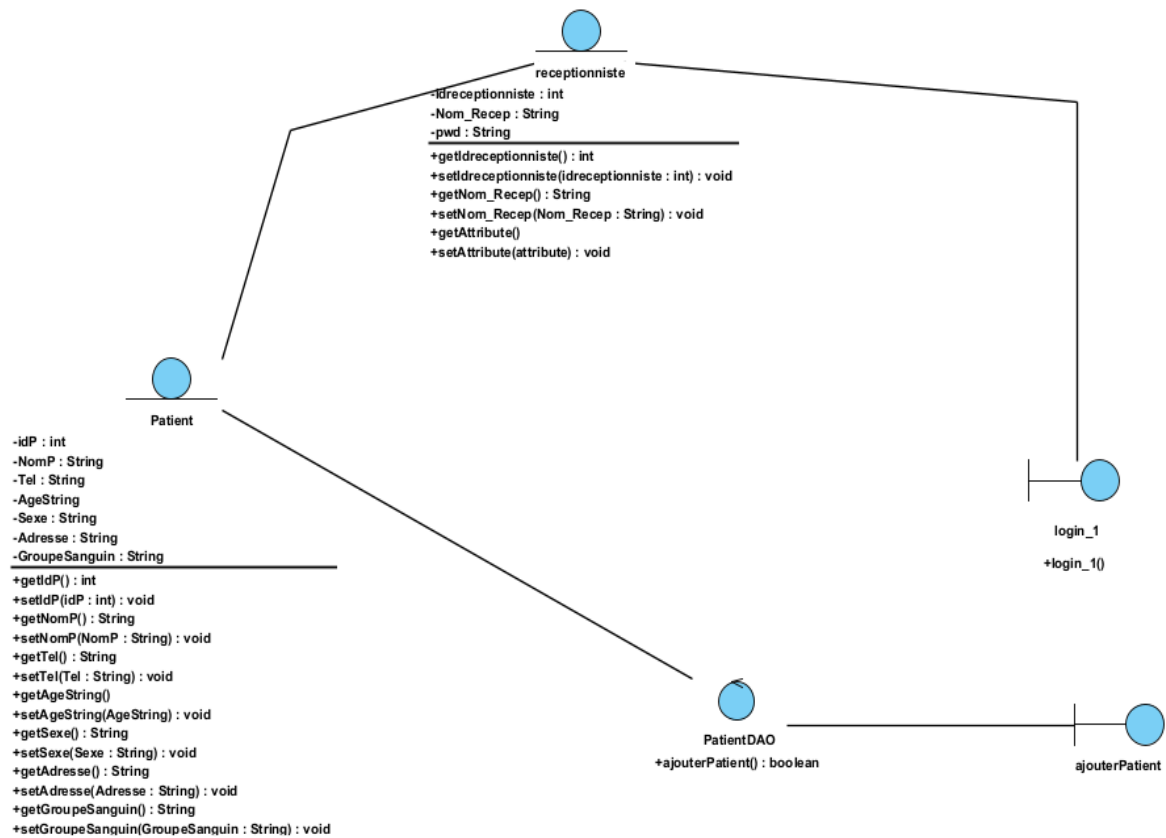


Figure16 : Diagramme de classes participantes Iteration2

c. itération 3 : Ajouter un diagnostic.

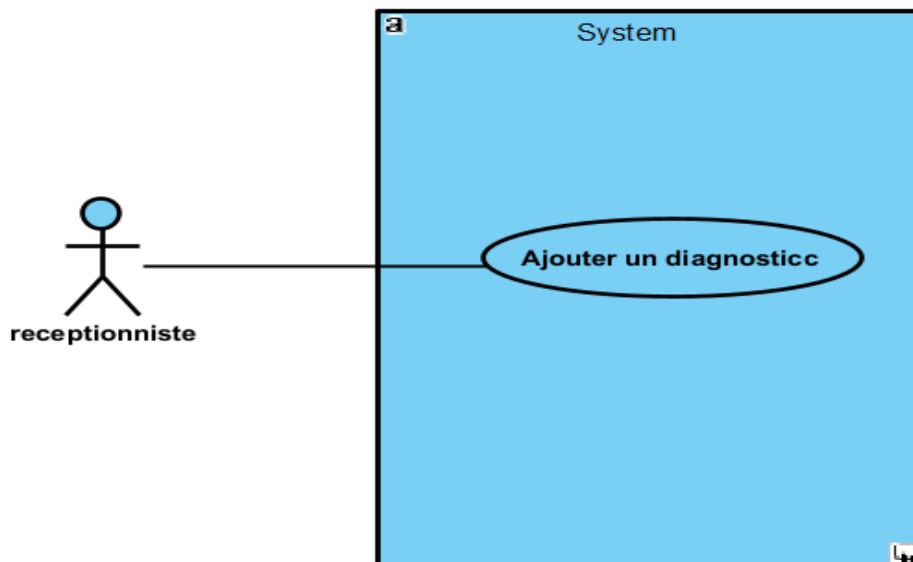


Figure17 : Diagramme de cas d'utilisation Iteration3

1) Diagramme de séquence :

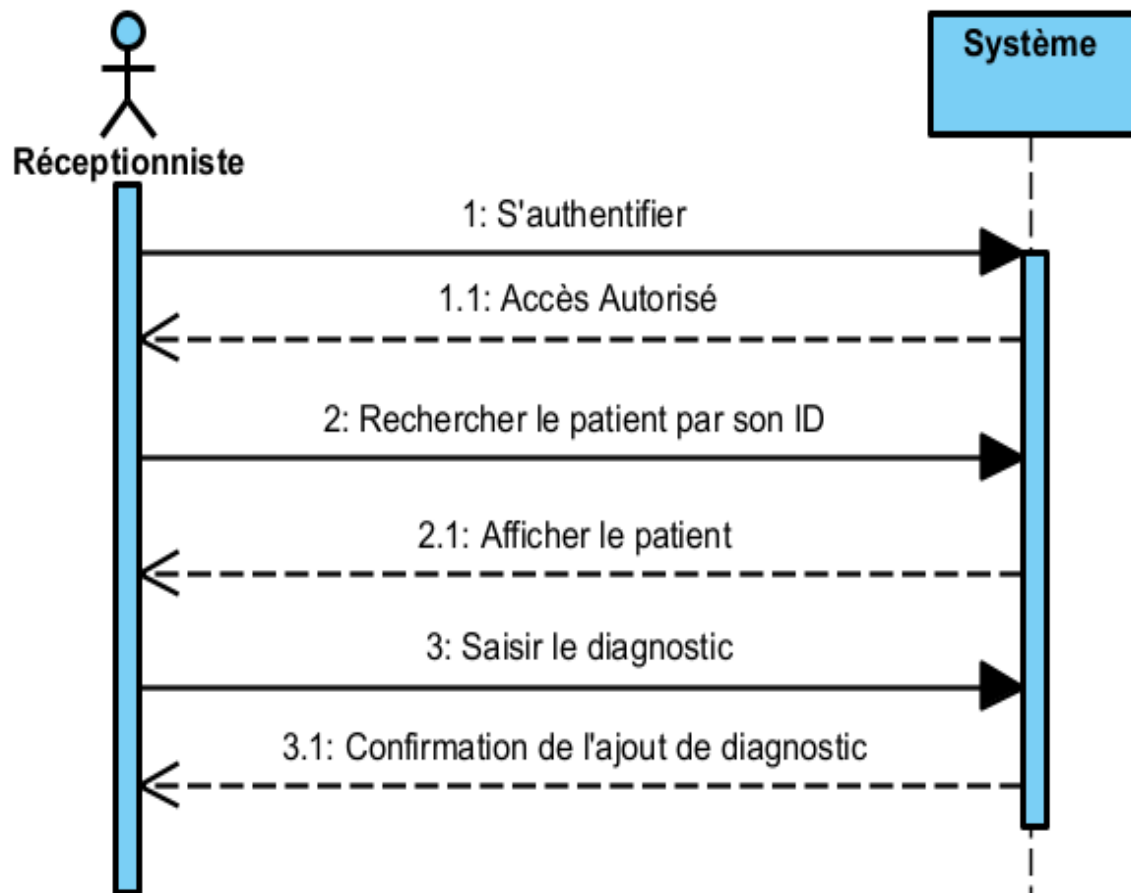
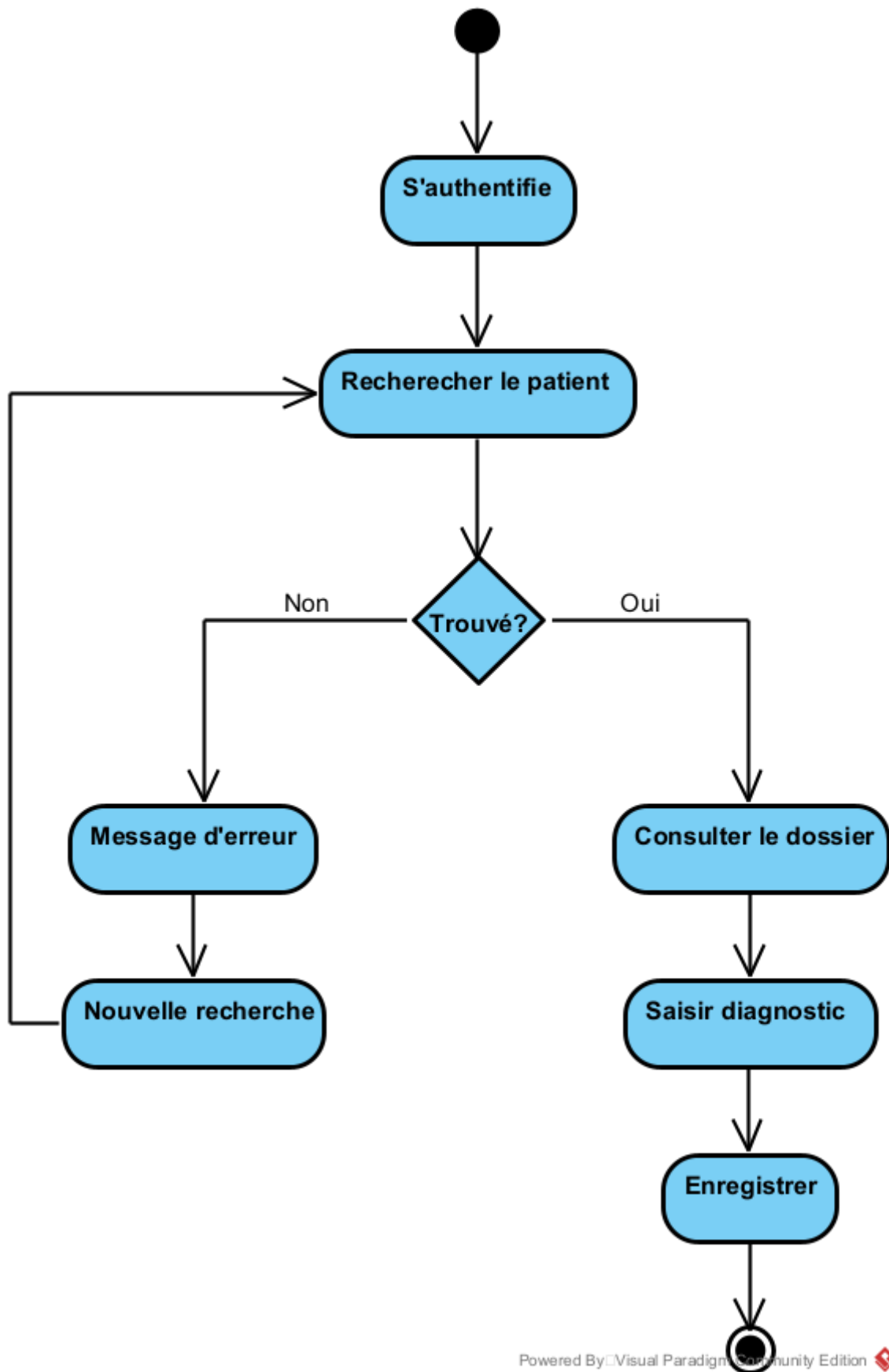


Figure18 : Diagramme de séquence Iteration3

2) Diagramme d'activité :



Powered By: Visual Paradigm Community Edition

Figure19 : Diagramme d'activité Iteration3

3) Diagramme de classes :

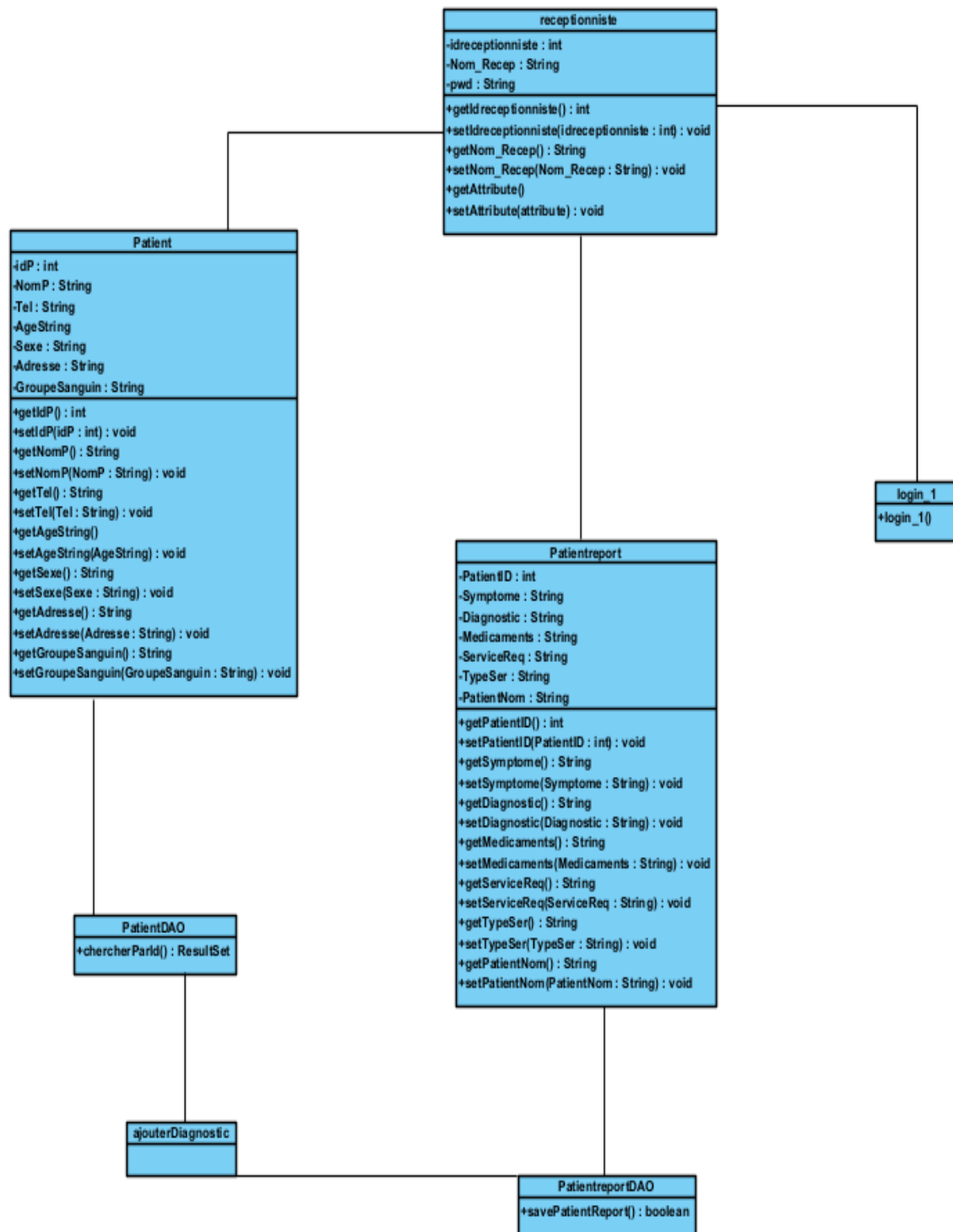


Figure20 : Diagramme de classes Iteration3

4) Diagramme de classes participantes :



Figure21 : Diagramme de classes participantes Iteration3

d. Itération 4 : Gérer l'historique de l'hôpital.

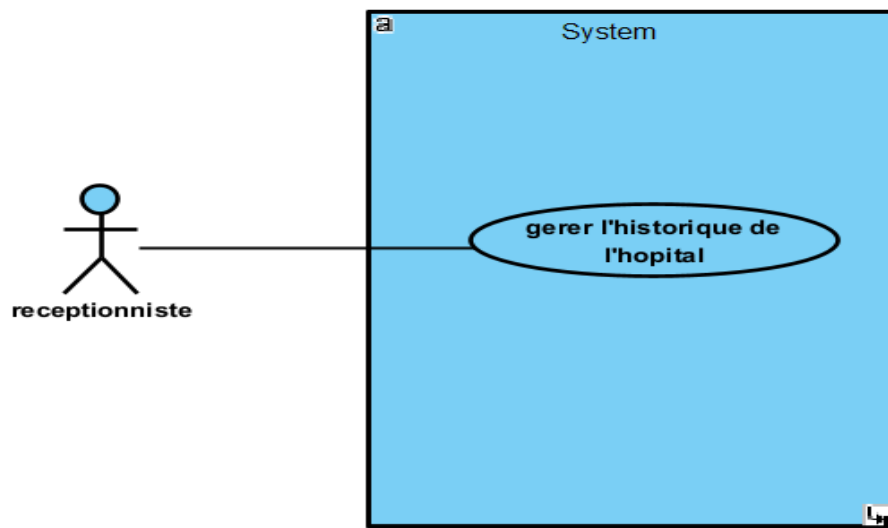


Figure22 : Diagramme de cas d'utilisation Iteration4

1) Diagramme de séquence :

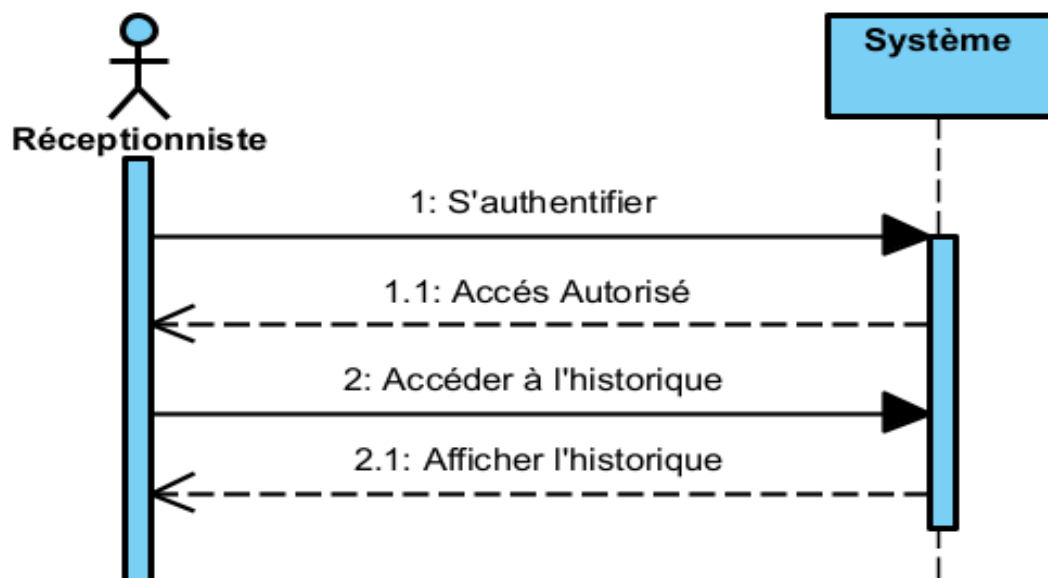


Figure23 : Diagramme de séquence Iteration4

2) Diagramme d'activité :

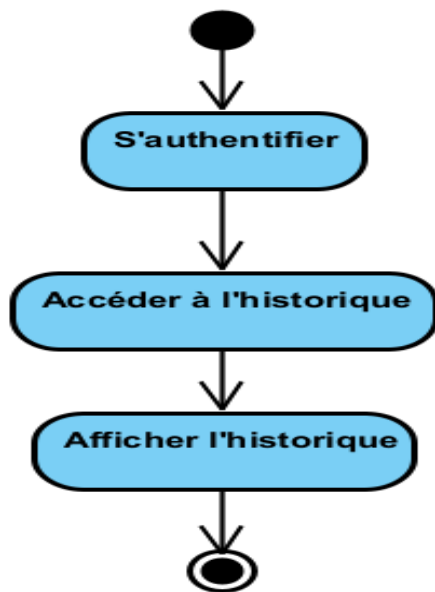


Figure24 : Diagramme d'activité Iteration4

3) Diagramme de classes:

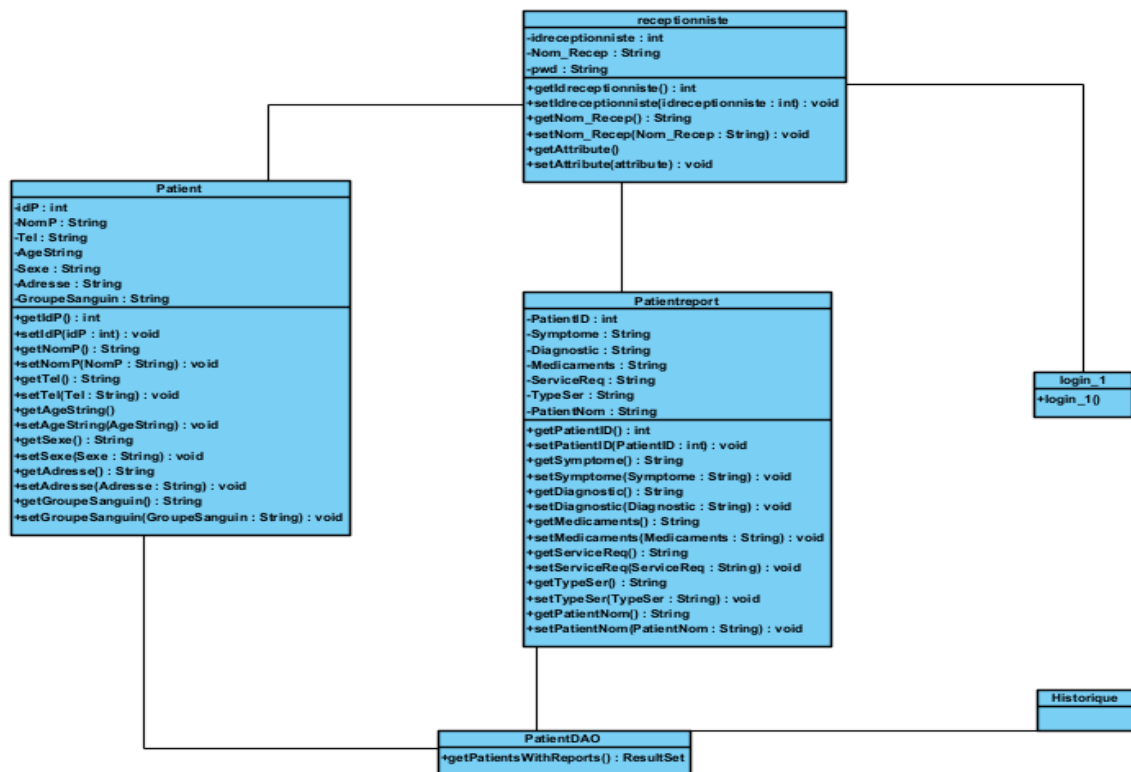


Figure25 : Diagramme de classes Iteration4

4) Diagramme de classes participantes :

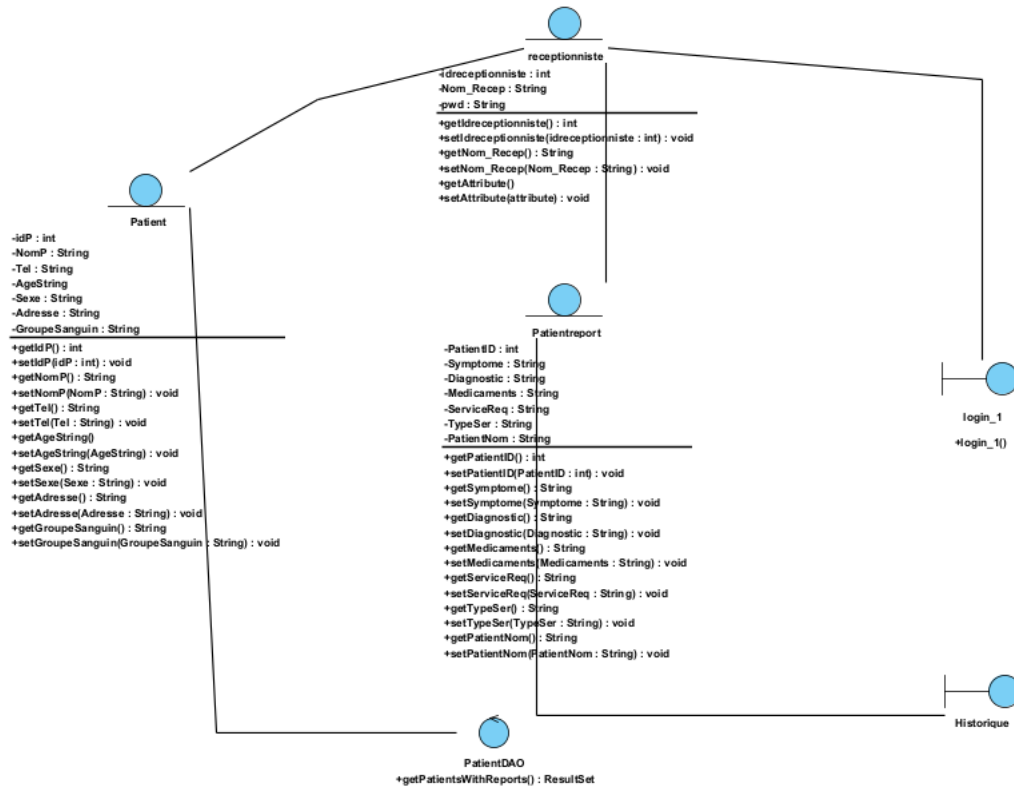


Figure25 : Diagramme de classes participantes Iteration4

e. Modifier les informations d'un un patient.

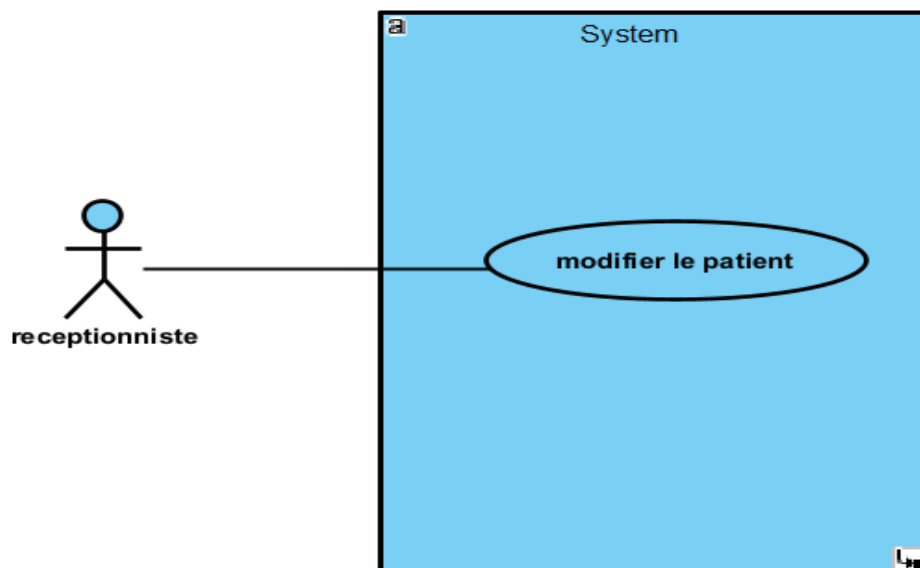


Figure26 : Diagramme de cas d'utilisation Iteration5

1) Diagramme de séquence :

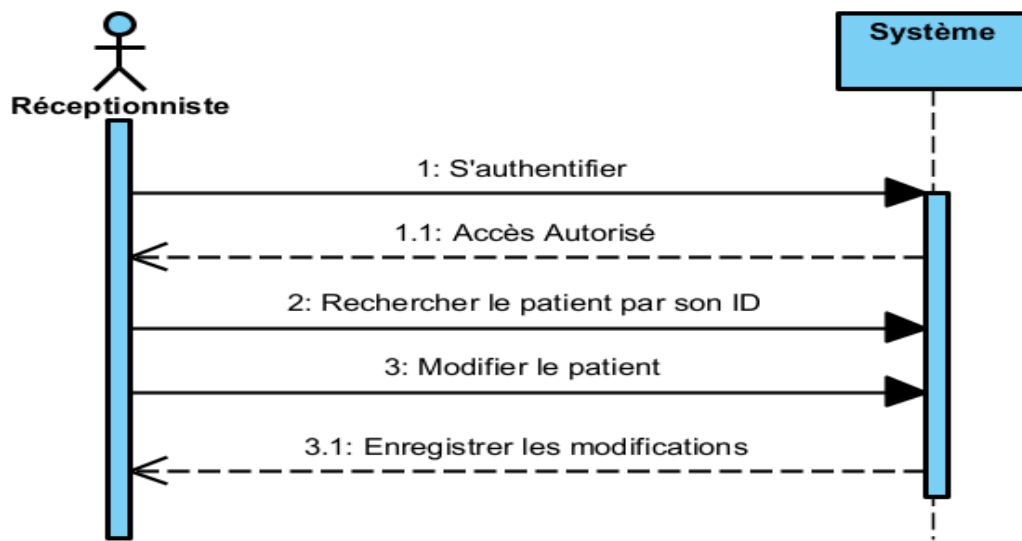


Figure26 : Diagramme de séquence Iteration5

2) Diagramme d'activité :

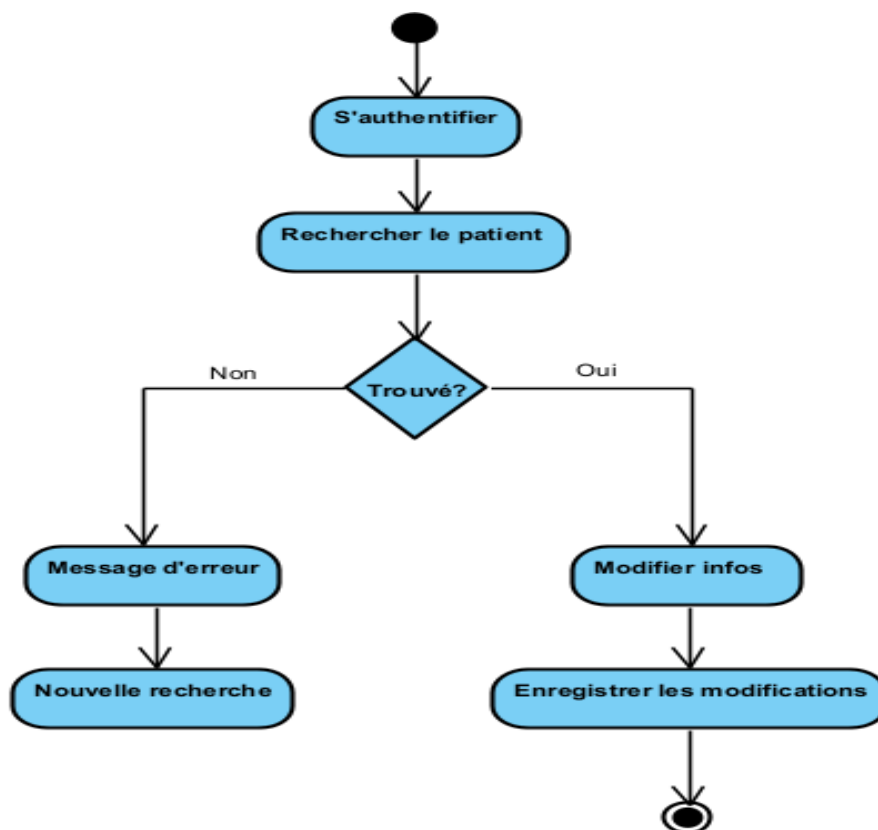


Figure27 : Diagramme d'activité Iteration5

3) Diagramme de classes :

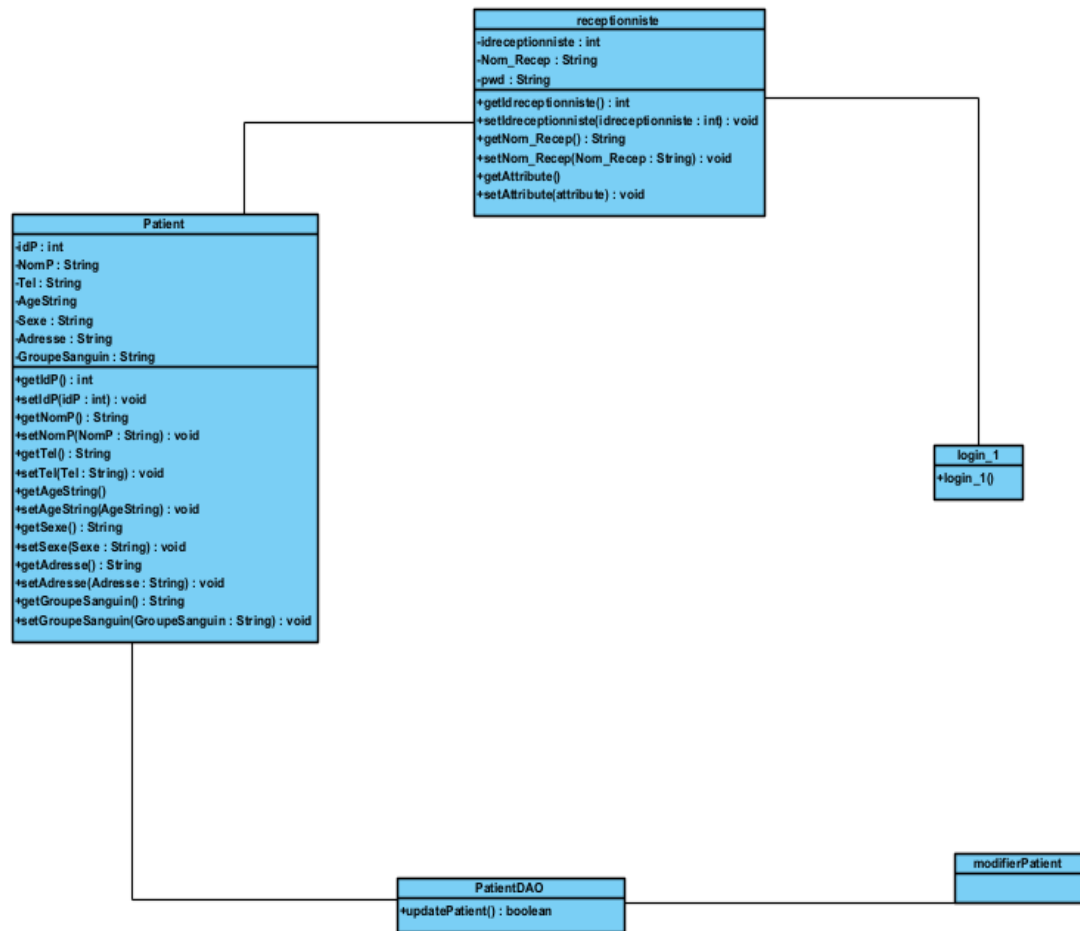


Figure28 : Diagramme de classes *Iteration5*

4) Diagramme de classes participantes :

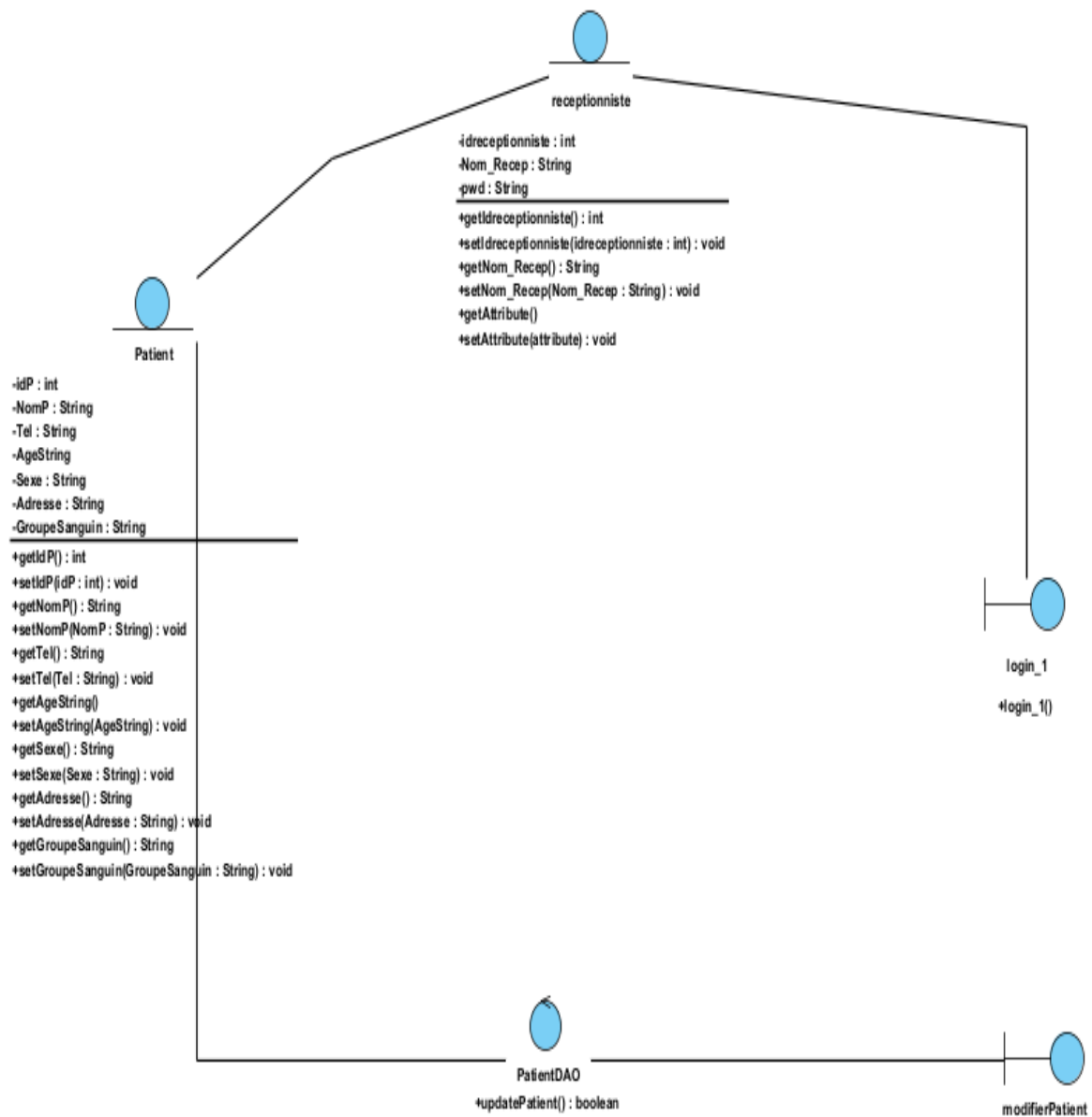


Figure29: Diagramme de classes participantes Iteration5

IV. 🔍 Réalisation :

1. Technologies utilisées :



Apache NetBeans : est un environnement de développement intégré (IDE) open-source, principalement utilisé pour le développement d'applications Java. Il permet aux développeurs de créer, compiler, exécuter, déboguer et tester des programmes dans un environnement unifié et convivial.

Figure 30 : NetBeans



Figure 31 : Visual Paradigm

Visual Paradigm est un outil de modélisation UML (Unified Modeling Language) professionnel utilisé pour concevoir, documenter et visualiser l'architecture des systèmes logiciels. Il aide les développeurs, analystes et architectes à mieux comprendre, planifier et structurer leurs projets.

Il prend en charge plusieurs types de diagrammes, notamment :

- Diagrammes de classes
- Diagrammes de cas d'utilisation
- Diagrammes de séquence
- Diagrammes d'activités
- Et bien d'autres...



MySQL : est un système de gestion de base de données relationnelle (SGBDR) open-source, basé sur le langage SQL (Structured Query Language). Il permet de créer, gérer, interroger et manipuler des bases de données de manière efficace et sécurisée.

Figure 32 : MySQL

2. Page de connexion du réceptionniste :

Cette interface est une fenêtre graphique de connexion utilisateur, réalisée avec Java Swing destinée à restreindre l'accès à une application. Elle contient le boutons suivants:

- **Se Connecter** : Bouton avec une icône verte (check) et le texte "Se Connecter". Permet d'envoyer les informations d'identification pour vérification.
- **Fermer** : Bouton avec une icône rouge (croix) et le texte "Fermer". Sert à quitter l'application ou fermer la fenêtre.

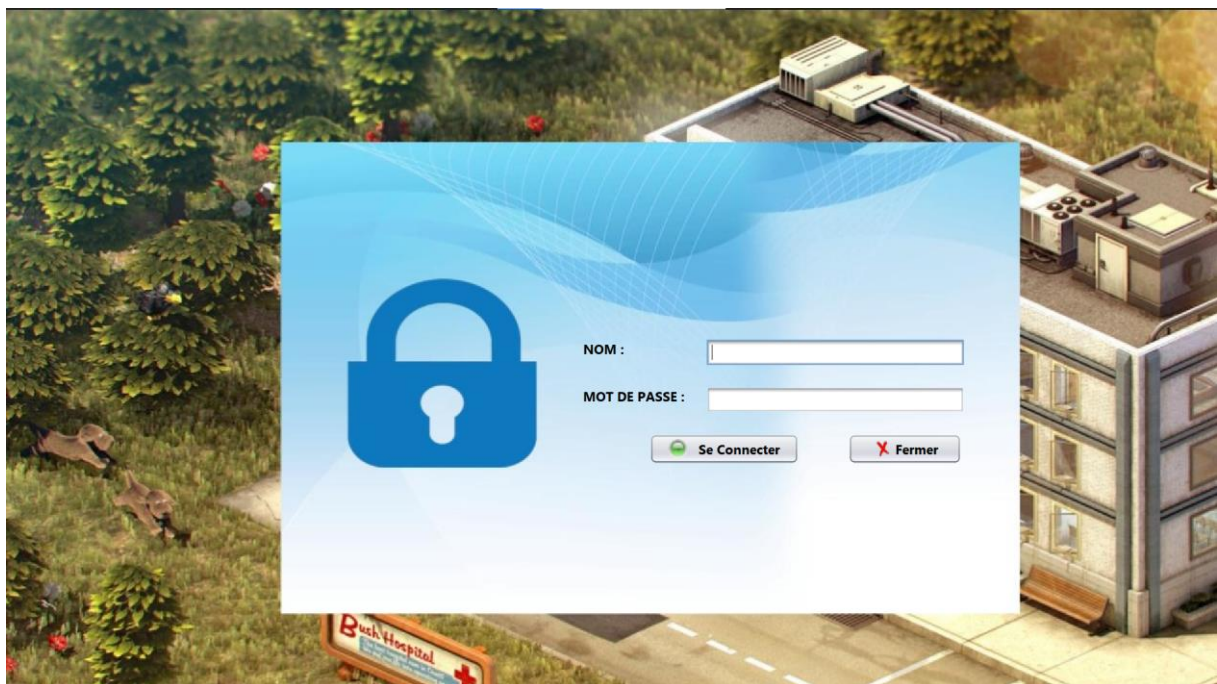


Figure 33 : Page de connexion de réceptionniste

Si les informations sont incorrectes, une boîte de dialogue s'affiche : "Le Nom ou le Mot de Passe est incorrect."

Cela montre une bonne gestion des erreurs d'authentification.

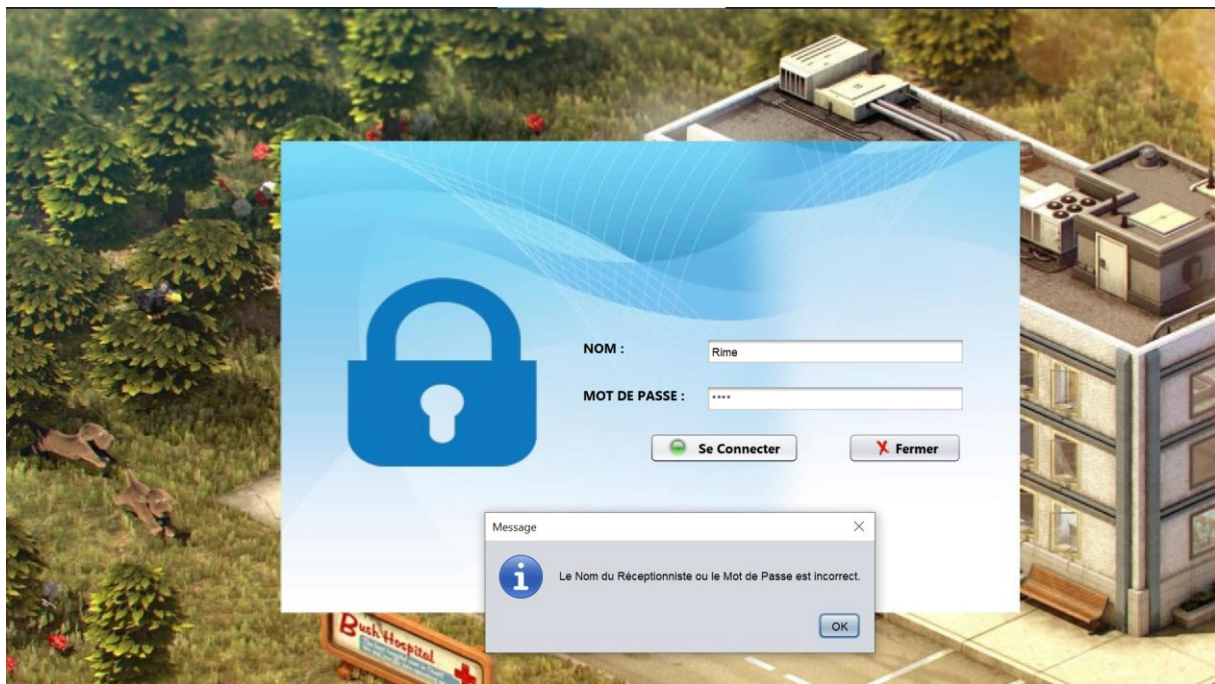


Figure 34 : Le message d'erreur

3. Page d'accueil :

Chaque bouton ouvre une interface spécifique :



Figure 35 : Page d'accueil

Buton 1 : Ajouter patient

Ce bouton sert à ajouter un nouveau patient



Figure 36 : Page d'ajout du patient

On ajout le nouveau patient et ,on clique sur le bouton enregistrer qui indique que l'enregistrement est effectué avec succès.

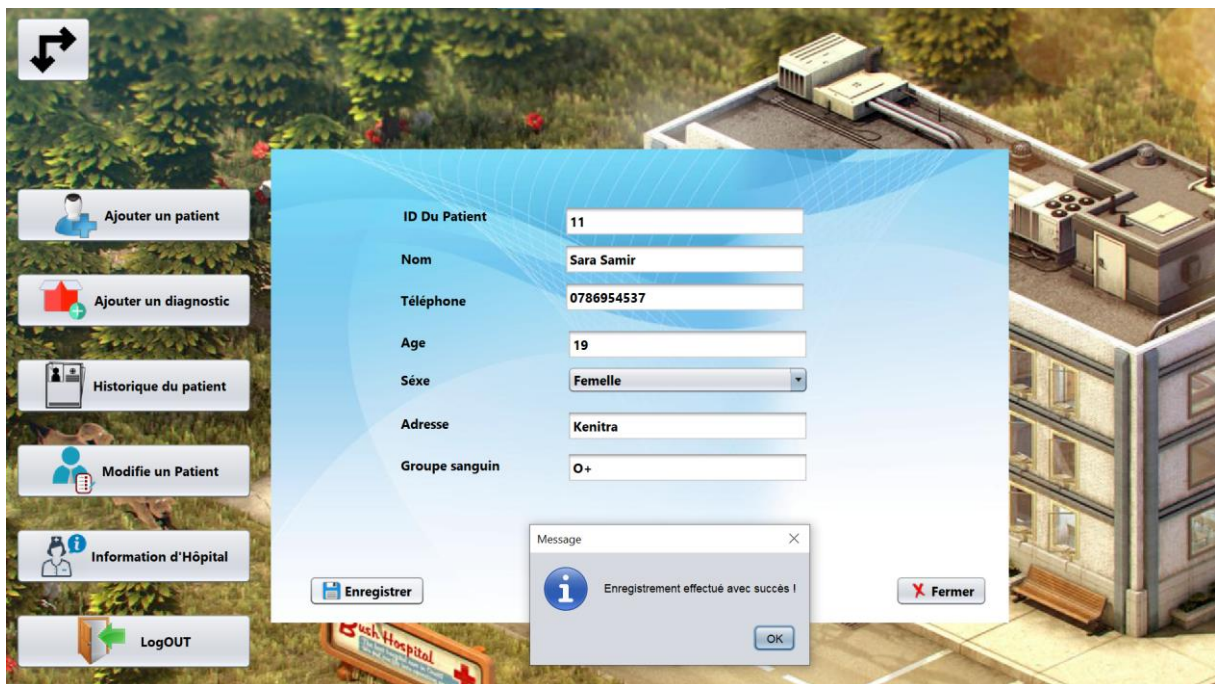


Figure 37 : l'enregistrement est effectué avec succès

Buton 2 : Ajouter diagnostic

Ce bouton sert à ajouter un diagnostic

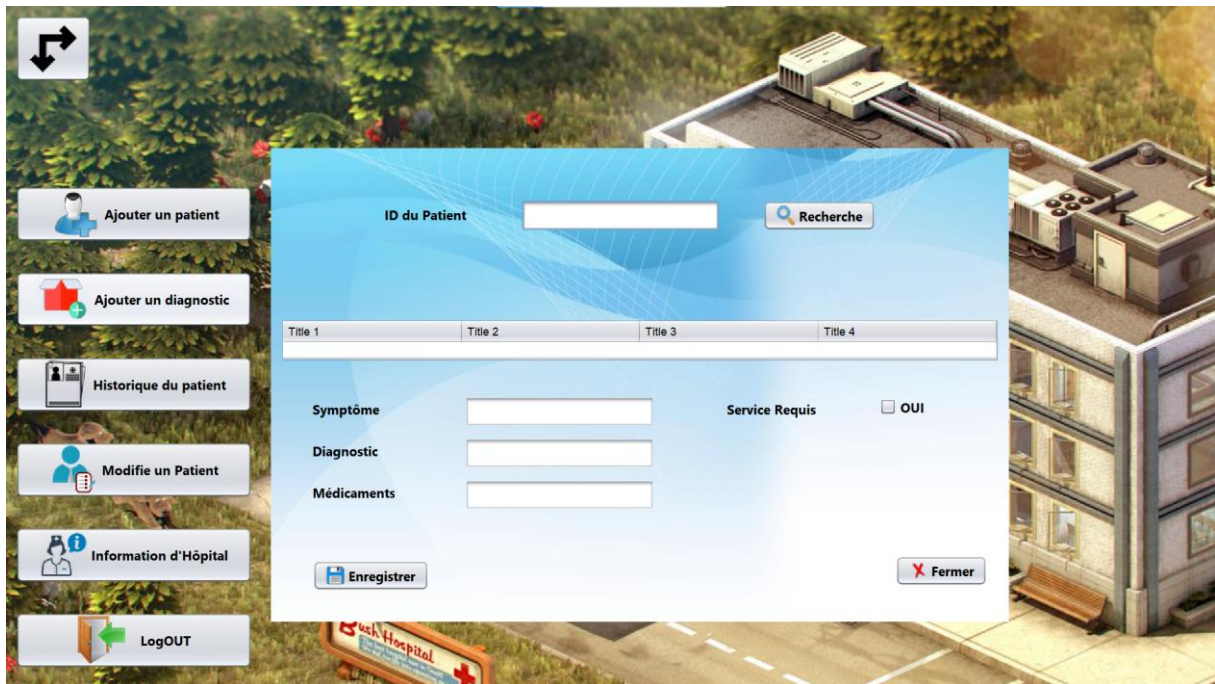


Figure 38 : Ajouter diagnostic

On recherche par ID et on Ajoute un diagnostic, puis on clique sur le bouton enregistrer qui indique que l'enregistrement est effectué avec succès.

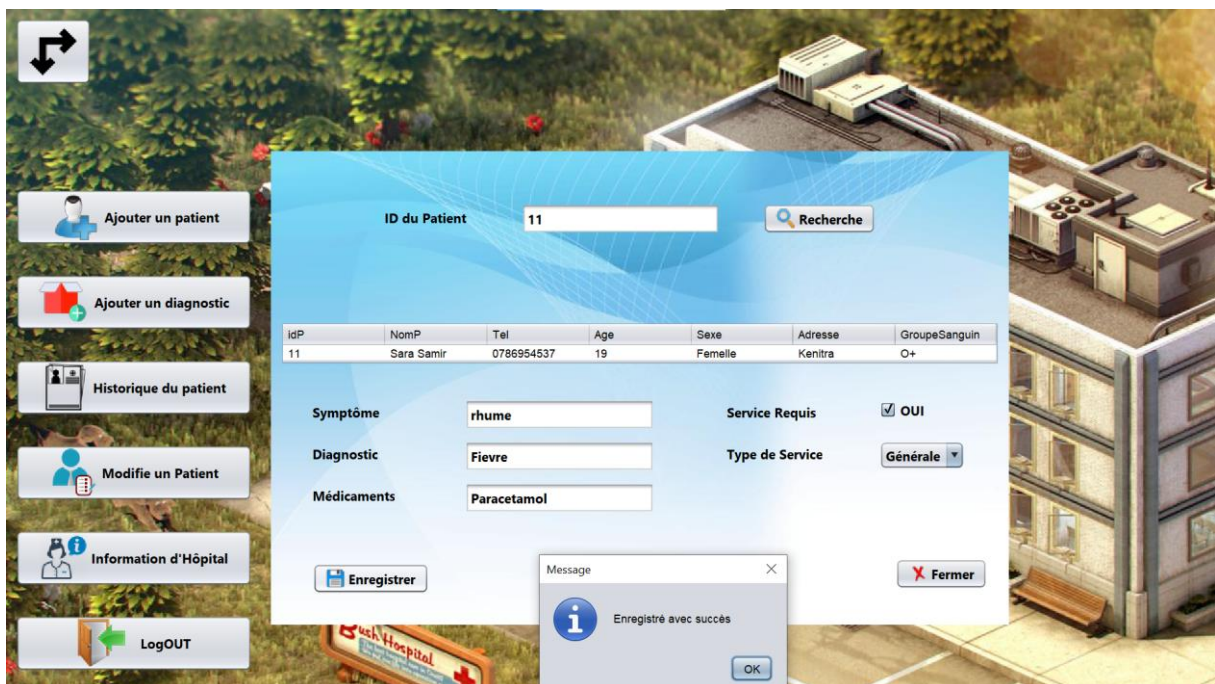


Figure 39 : l'enregistrement est effectué avec succès

Buton 3 : L'historique

Les données enregistrées sont affichées donc sur le tableau

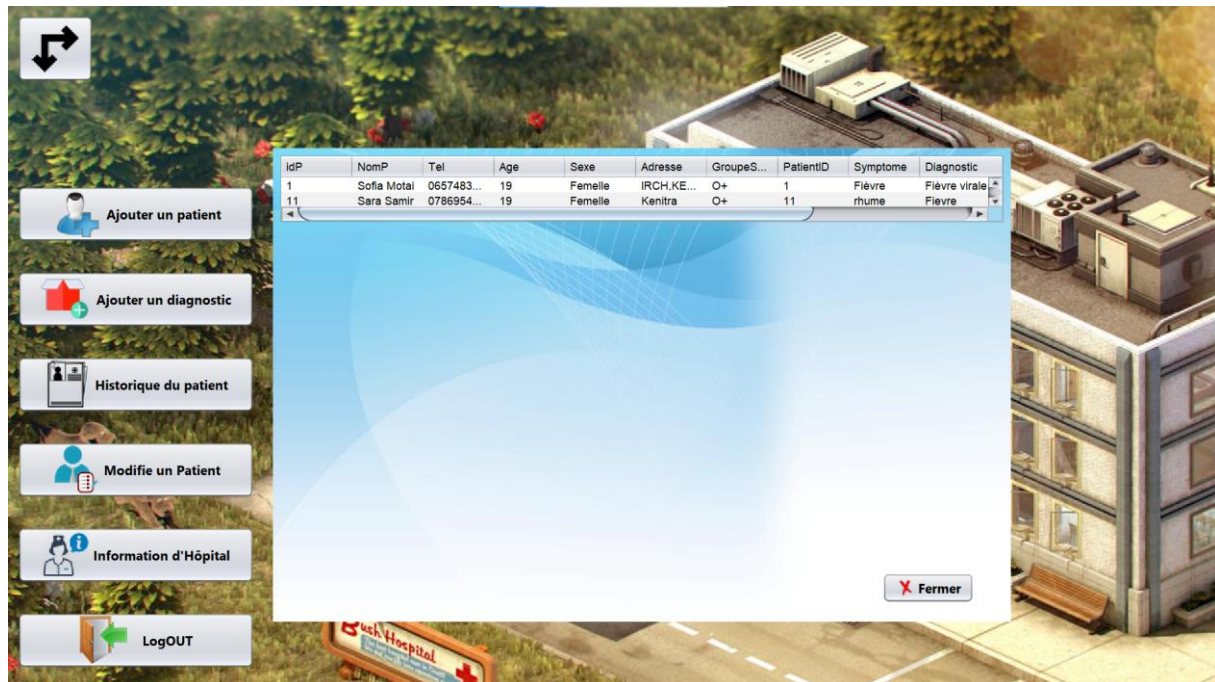


Figure 40 : l'historique

Buton 4 : Modifier patient

Ce bouton sert à modifier un patient

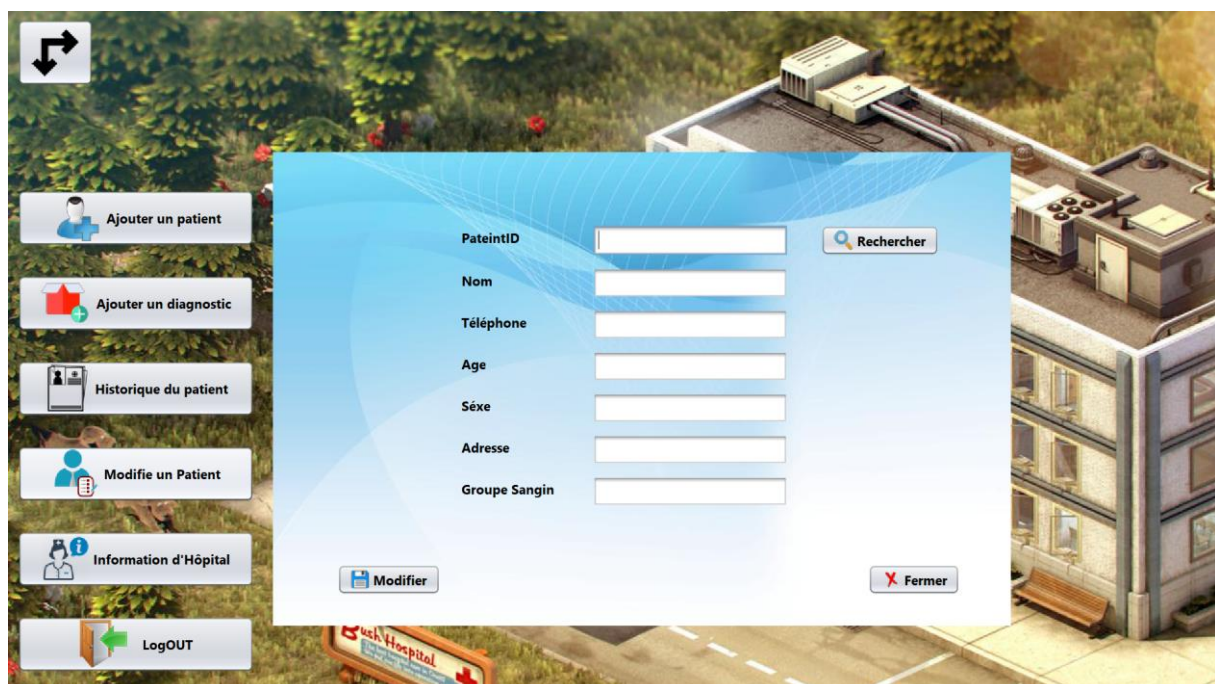


Figure 41 : Page de modification du patient

On recherche par ID et on modifie le patient, puis on clique sur le bouton modifier qui indique que la modification est effectuée avec succès.

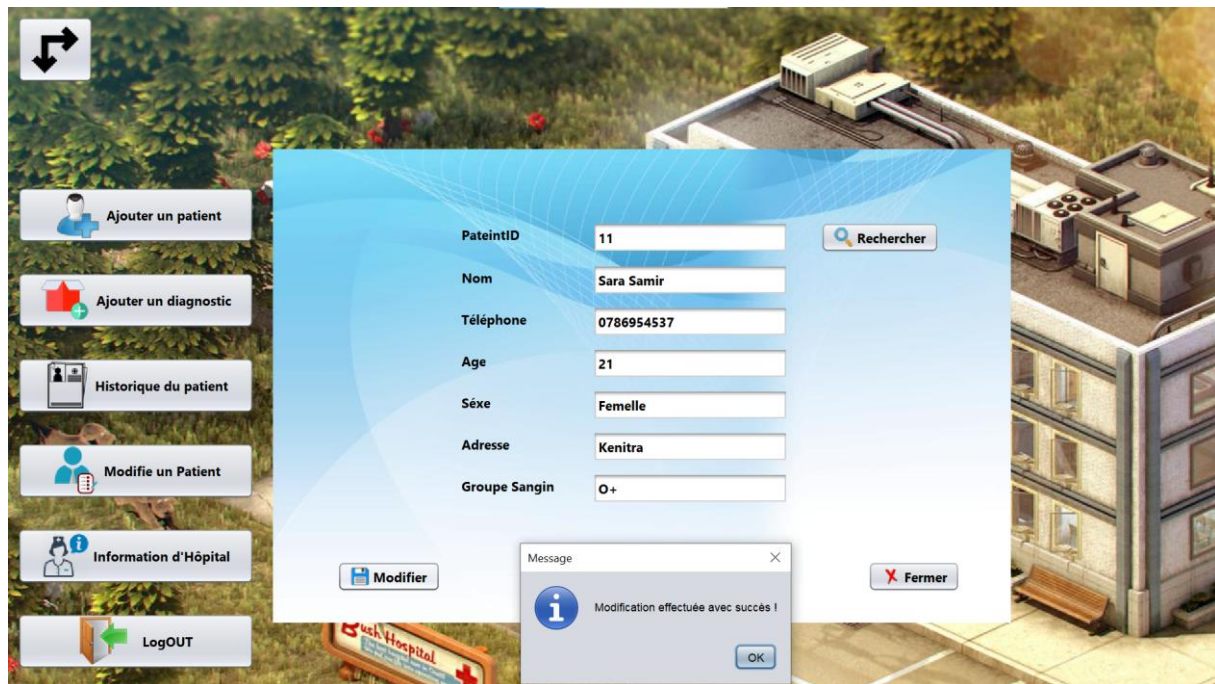


Figure 42 : la modification est effectuée avec succès.

Buton 5 : Information d'hôpital



Figure 43 : Information d'hôpital

Buton 6 : Déconnexion

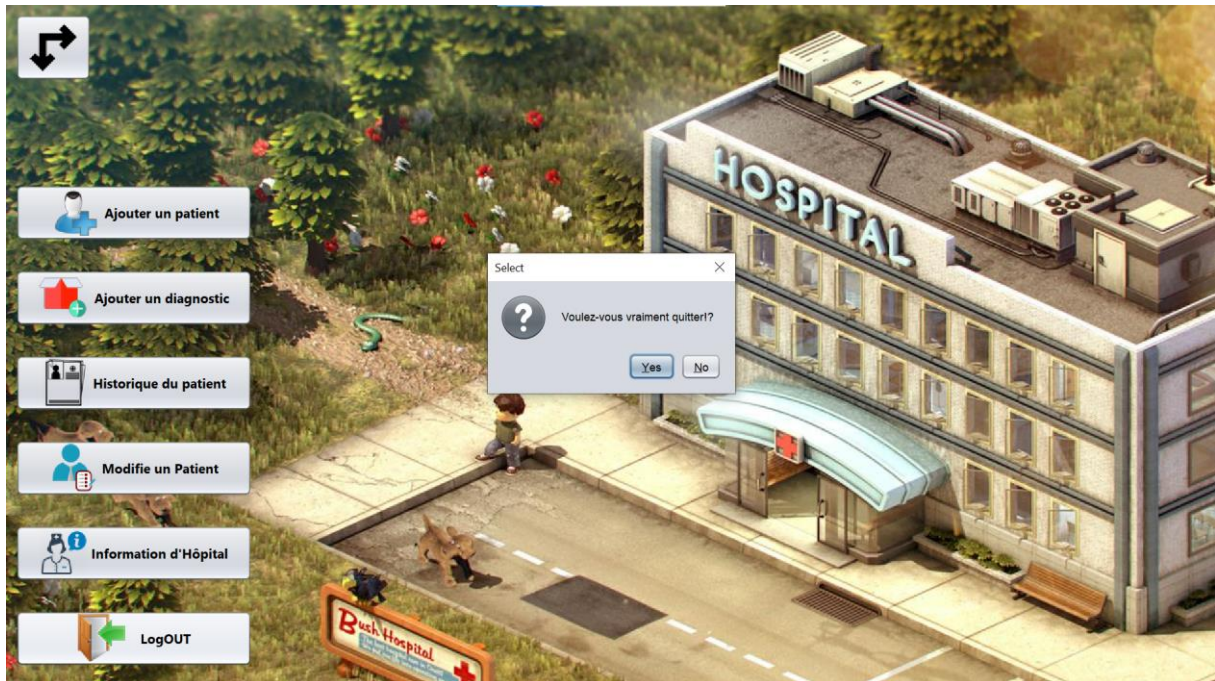


Figure 44 : Déconnexion

4. Test Unitaire JUnit :

4.1. LoginTest :

Structure standard des tests JUnit:

- Des méthodes de configuration (setUpClass, setUp, etc.) qui sont vides mais pourraient être utilisées pour préparer l'environnement de test
- Trois méthodes de test annotées avec @Test

Test de connexion valide:

- Vérifie que la méthode verifierConnexion retourne true avec les identifiants "Rime" et "123"
- Utilise assertTrue pour confirmer que la connexion réussit

Test de connexion invalide:

- Vérifie que la méthode verifierConnexion retourne false avec des identifiants incorrects "Admin" et "456"
- Utilise assertFalse pour confirmer que la connexion échoue

Test de la méthode main:

- Test simple de la méthode main, sans vérifications spécifiques
- Comme indiqué dans le commentaire, ce test semble être facultatif

Pour que ces tests fonctionnent correctement, vous devez avoir une classe `login_1` qui implémente:

- Une méthode `verifierConnexion(String nom, String motDePasse)` qui vérifie les identifiants
- Une méthode `main`

Ces tests suggèrent que votre système d'authentification accepte au moins l'utilisateur "Rime" avec le mot de passe "123", et rejette d'autres combinaisons comme "Admin"/"456".

4.2. ConnexionTest :

Structure standard JUnit (comme dans le test précédent):

Des méthodes de configuration vides (`setUpClass`, `setUp`, etc.)

Test de la méthode `getCon`:

La méthode testée est `Connexion.getCon()` qui semble être une méthode statique retournant un objet `Connection` (de la librairie `java.sql`)

Le test vérifie que:

- La connexion retournée n'est pas null (`assertNotNull`)
- La connexion peut être fermée proprement après utilisation (dans un bloc `try-catch`)

Ce test indique que la classe `Connexion` devrait avoir:

- Une méthode statique `getCon()` qui retourne un objet `Connection` à votre base de données
- Cette méthode devrait établir une connexion fonctionnelle à la base de données

Pour que ce test fonctionne, vous aurez besoin d'une implémentation de la classe `Connexion` qui établit correctement la connexion à votre base de données

4.3. PatientDAOTest :

Structure standard JUnit avec les méthodes de configuration habituelles

Test d'ajout de patient (`testAjouterPatient`):

Crée un nouveau patient avec des données spécifiques (ID "11", nom "Amal", etc.)

Teste si la méthode ajouterPatient retourne true, indiquant un ajout réussi

Test de recherche par ID (testChercherParId):

- Recherche un patient avec l'ID "5"
- Vérifie que le ResultSet n'est pas null et contient un patient avec l'ID correspondant

Test de mise à jour d'un patient (testUpdatePatient):

- Teste la mise à jour des informations d'un patient avec l'ID "11"

Test des patients avec rapports (testGetPatientsWithReports):

- Vérifie que la méthode retourne un ResultSet non-null qui contient au moins une ligne

Test du modèle de table (testGetPatientsWithReportsTableModel):

- Vérifie que la méthode retourne un TableModel non-null qui contient au moins une ligne

Ces tests suggèrent que la classe PatientDAO devrait implémenter :

- ajouterPatient(Patient): pour ajouter un patient à la base de données
- chercherParId(String): pour trouver un patient par son ID
- updatePatient(...): pour mettre à jour les informations d'un patient
- getPatientsWithReports(): pour obtenir les patients qui ont des rapports
- getPatientsWithReportsTableModel(): pour obtenir un modèle de table pour affichage dans une interface graphique

4.4. PatientreportTest :

Structure standard JUnit avec les méthodes de configuration habituelles, toutes vides.

Test de sauvegarde de rapport patient (testSavePatientReportValidData):

Ce test vérifie la méthode savePatientReport qui semble enregistrer un rapport médical pour un patient

Paramètres testés :

- ID patient: "10"
- Symptôme: "Fièvre"
- Diagnostic: "Grippe"
- Médicaments: "Paracétamol"
- Service requis: "YES"
- Type de service: "URgence"